Government Procurement Anomaly Detection System

IntegriScan: An ML-Powered Tool to Flag Irregularities in Public Spending Data

A Capstone Project Report

Submitted by:

Edrissa Sarr - C799840

Master of Science in Information Technology (MSIT)

Supervised by:

Dr. Sirisha Pavuluri

Department of Computer Science and Information Technology

University of the People, Pasadena, CA 91101, United States

MSIT 5910: Capstone Project | AY2027-T1, October 2026

ABSTRACT

Government procurement is one of the most corruption-prone areas of public administration globally, with financial irregularities costing governments an estimated 10–25% of contract values annually. The Organization for Economic Co-operation and Development (OECD) estimates these losses amount to hundreds of billions of dollars in misallocated public funds each year (OECD, 2025). Traditional manual auditing approaches cannot scale to the volume of contracts processed by modern public agencies, creating a critical oversight gap that automated machine learning approaches are uniquely positioned to address.

This Capstone project designed, implemented, tested, and documented the Government Procurement Anomaly Detection System, branded as IntegriScan, and operating under the motto Follow the money. Flag the risk: a machine learning-powered web application that applies Isolation Forest and z-score anomaly detection algorithms to open government procurement data from the United States government's USASpending.gov portal. The system enables non-technical government auditors to identify and prioritize suspicious contracts for investigation through an interactive Streamlit dashboard with explainable artificial intelligence (XAI) risk factor descriptions, role-based access control, and an auditor actionability feedback mechanism.

The system was developed over eight units using a layered five-module architecture comprising a Python-based ETL pipeline, a PostgreSQL 18 relational database with five tables

and twenty-six indexes, a scikit-learn Isolation Forest machine learning detection module, an XAI engine generating plain-language risk explanations, and a Streamlit web dashboard with role-based access control enforced at both the application and database layers. The ETL pipeline processed 50,000 rows from the 9.3 gigabyte FY2023 Department of Defense contracts dataset in 62.6 seconds at a throughput of 798 records per second. The machine learning module trained an Isolation Forest model with a contamination parameter of 0.05, achieving a recall of 64.5% on 31 synthetic anomaly labels and generating 2,397 unique anomaly flags stored across two detection categories: 1,776 year-end cluster flags and 621 price outlier year-end flags.

A comprehensive testing framework comprising 66 pytest tests across four levels: unit, integration, system, and acceptance, achieved a 100% pass rate in 7.68 seconds. All seven success criteria from the original Capstone proposal were verified through executable acceptance tests, providing objective and auditable confirmation that the system meets its design objectives. The system is accompanied by a complete documentation package including an Ethical Use and Data Governance Policy identifying seven sources of algorithmic bias, a Database Data Dictionary documenting all five tables and three views, a Project Management Log tracking all eight development units, and a post-Capstone product roadmap defining fifteen enhancements across five development phases.

Keywords: procurement anomaly detection, Isolation Forest, z-score, explainable artificial intelligence, government accountability, machine learning, Streamlit, role-based access control, PostgreSQL, USASpending.gov, public sector analytics.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Dr. Sirisha Pavuluri, my Capstone supervisor at the University of the People, whose timely and substantive feedback across all eight units of this project provided essential guidance that shaped the technical design, academic rigor, and professional quality of this report. The emphasis on connecting implementation decisions to course learning outcomes and program objectives encouraged me to think beyond the code itself and consider the broader ethical, organizational, and governance dimensions of building AI-powered tools for public accountability.

I am deeply grateful to the six peer reviewers: James Scarberry, Philipp Bomers, Chris Hazell, Dennis Morales, Tin Lok Ho, Khadheraa Muaydh Mohammed Al Hajebi, and Jehanzaib Bhatti, whose substantive and technically rigorous feedback across the Unit 1 and Unit 3 discussion forums generated twelve concrete design commitments that materially improved the IntegriScan system. Their contributions directly shaped the explainable risk factor engine, the auditor actionability feedback mechanism, the log-normal synthetic anomaly injection methodology, the contamination threshold tuning approach, and the governance controls for administrator accountability. All twelve commitments were implemented and verified through the 66-test suite.

I am also grateful to the University of the People for providing access to the course readings and LIRN library resources that grounded this project in the existing literature on software engineering, machine learning, cybersecurity, and ethics. The open-source tools and publicly available datasets used throughout this project- Python, PostgreSQL, scikit-learn, Streamlit, and USASpending.gov- reflect a commitment to zero-cost, reproducible, and community-driven technology that aligns with the University's mission of accessible, high-quality global education.

Finally, I dedicate this work to my love, Adika, my dear mom, my late dad, my employer, Semlex Côte d'Ivoire, and to the people of The Gambia, in whose service the governance transparency and accountability tools demonstrated in this project will ultimately be deployed through the IntegriScan Technologies product roadmap. The vision of using data-driven oversight to follow the money and flag the risk in public procurement is not merely an academic exercise; it is a contribution to the broader project of democratic accountability and good governance that the world urgently needs.

UNIVERS
OF THE PE(

TABLE OF CONTENTS

Abstract 2

Acknowledgements 4

Table of Contents 6

List of Tables 9

List of Figures 10

List of Abbreviations and Symbols 11

Chapter 1: Introduction 12

1.1 Background and Motivation 12

1.2 Problem Statement 14

1.3 Project Objectives 15

1.4 Scope and Boundaries 16

1.5 Significance of the Project 17

1.6 Report Structure 18

Chapter 2: Literature Review 19

2.1 Procurement Fraud and Anomaly Detection 19

2.2 Machine Learning Approaches for Anomaly Detection 20

2.3 Explainable AI in Audit Decision Support Systems 22

2.4 Security and Ethical Frameworks for AI Systems 23

2.5 Open Government Data and Procurement Transparency 24

2.6 Gap Analysis: What IntegriScan Contributes 25

Chapter 3: System Design and Methodology 27

3.1 Research Design and Methodology 27

3.2 System Architecture 28

3.3 Database Design 30

3.4 ETL Pipeline Design 31

3.5 Machine Learning Module Design 32

3.6 Dashboard Design 33

3.7 Security Design 34

3.8 Ethical Design 35

Chapter 4: Results and Evaluation 36

4.1 ETL Pipeline Results 36

4.2 ML Detection Results 38

4.3 Testing Results 40

4.3.1 Key Code Snippets with Outputs 43

4.4 Dashboard Integration Results 47

4.5 Version Control Milestones 49

Chapter 5: Discussion 51

5.1 Interpretation of Results 51

5.2 Evaluation Metrics Summary 53

5.3 Deployment Approach 55

5.4 Limitations 56

5.5 Recommendations for Future Research 57

Chapter 6: Conclusion and Future Work 59

6.1 Summary of Main Findings 59

6.2 Contributions to the Field 60

6.3 Lessons Learned 62

6.4 Future Work and Product Roadmap 63

References 66

Appendix A: System Architecture Diagram 68

Appendix B: Database Schema Summary 70

Appendix C: Complete Test Suite Output 71

Appendix D: Key Code Sections 73

LIST OF TABLES

Table 1.1. Course Integration Map: 9 MSIT Courses Applied in IntegriScan 16

Table 1.2. Functional and Non-Functional Requirements Summary 17

Table 3.1. Technology Stack and Justification 29

Table 3.2. Database Tables, Row Counts, and Purpose.....31

Table 3.3. Machine Learning Features: Engineering and Rationale.....32

Table 4.1. ETL Pipeline Test Run Results.....36

Table 4.2. ML Detection Module Results.....38

Table 4.3. Complete Test Suite: 66 Tests Across 4 Levels 40

Table 4.4. Acceptance Criteria Verification: All 7 Criteria Met 43

Table 4.5. GitHub Version Control Milestones.....49

Table 5.1. Quantitative Performance Metrics Summary 53

Table 5.2. Qualitative Usability Evaluation: 5 Criteria 54

Table 5.3. Deployment Options Comparison 55

Table 6.1. Post-Capstone Product Roadmap: 15 Enhancements 64

LIST OF FIGURES

Figure 3.1. Five-Layer System Architecture....29

Figure 4.1. ETL Pipeline Output: 47,540 Records Loaded....37

Figure 4.2. ML Detection Module Output: 2,397 Flags....39

Figure 4.3. pgAdmin: Anomaly Flags Breakdown by Type....40

Figure 4.4. pytest Output: 34 Unit Tests Passed....41

Figure 4.5. pytest Output: All 66 Tests Combined....42

Figure D.1. Isolation Forest Code: ml_module.py....44

Figure D.2. Z-Score Detection Logic: ml_module.py....45

Figure D.3. Risk Explanation Function: ml_module.py....46

Figure 4.6. Dashboard Overview: Summary Metrics and Chart....47

Figure 4.7. Flagged Contracts: IF Score and Z-Score Side by Side....48

Figure 4.8. Explainable Risk Factors: XAI Panel Expanded....48

Figure 4.9. GitHub Release v1.0, v2.0, v3.0, & v4.0 Published....50

Figure 5.1. Actionability Rate Tracker: Pending Analyst Review....54

List of Abbreviations and Symbols

Abbreviation / Symbol	Definition
AI	Artificial Intelligence
API	Application Programming Interface
COTS	Commercial Off-The-Shelf
DB	Database
DoD	Department of Defense (United States)
ECOWAS	Economic Community of West African States
ETL	Extract, Transform, Load
FR	Functional Requirement
FY	Fiscal Year (US Federal: October 1 – September 30)
GDPR	General Data Protection Regulation (EU)
IF	Isolation Forest (machine learning algorithm)
KPI	Key Performance Indicator
ML	Machine Learning
MSIT	Master of Science in Information Technology
NAICS	North American Industry Classification System
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
OECD	Organization for Economic Co-operation and Development
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PLO	Program Learning Objective
RBAC	Role-Based Access Control
RMF	Risk Management Framework
SaaS	Software as a Service
SCM	Software Configuration Management
SQL	Structured Query Language
UI	User Interface
UX	User Experience
XAI	Explainable Artificial Intelligence
z-score	Standard score: deviation from mean in standard deviation units (sigma)

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Government procurement, the process by which public agencies acquire goods, services, and infrastructure from private vendors, represents one of the largest categories of public expenditure globally. The Organization for Economic Co-operation and Development estimates that government procurement accounts for approximately 12% of GDP in OECD member countries and up to 30% of GDP in developing economies, making it one of the most significant mechanisms through which public funds flow from government to the private sector (OECD, 2025). This scale of spending simultaneously makes procurement one of the highest-risk areas for financial irregularities, waste, and corruption. When procurement systems fail, through fraud, collusion, or negligence, the consequences extend far beyond financial loss: public infrastructure is degraded, essential services are underfunded, and public trust in government institutions is eroded.

The OECD further estimates that corruption in public procurement adds an average of 10–25% to the cost of government contracts, translating to hundreds of billions of dollars in misallocated public funds annually (OECD, 2025). In developing economies where government budgets are already constrained, and institutional oversight mechanisms are nascent, the proportional impact is even more severe. A procurement fraud pattern that costs a high-income



government agency 15% of contract value may cost a developing-country agency 40% of its annual capital budget for a critical public service.

Traditional approaches to procurement oversight rely on manual auditing, a process that is fundamentally unable to scale to the volume and complexity of modern government contracting. A single large federal agency may award tens of thousands of contracts annually, each requiring evaluation against dozens of potential irregularity indicators: inflated pricing relative to market rates, unusual award timing patterns, duplicate or related vendor registrations, contract splitting to circumvent approval thresholds, and single-source awards that bypass competitive bidding requirements. Manual auditors, constrained by time and resources, can realistically examine only a small fraction of these contracts, typically selected through random sampling or tip-based investigation, leaving the vast majority unreviewed and vulnerable to undetected irregularities.

The emergence of open government data initiatives, particularly the USASpending.gov portal in the United States, which publishes structured, standardized records of all federal contract awards, has created an unprecedented opportunity to apply machine learning methods to government procurement oversight at scale. These publicly available datasets contain millions of records with consistent schema, enabling automated pattern detection that was previously impossible without direct access to government internal systems. The IntegriScan project was designed to demonstrate how this opportunity can be realized using open-source tools, publicly

available data, and a human-in-the-loop design that keeps government auditors in control of all investigative decisions.

1.2 Problem Statement

Government procurement systems often operate in organizational silos with limited automated oversight and manual auditing processes that are resource-intensive, inconsistent in their coverage, and unable to scale with the volume of contracts processed by modern public agencies. Five specific problem areas were identified through the literature review and confirmed by examination of the USASpending.gov FY2023 Department of Defense contracts dataset.

First, price inflation: contracts awarded significantly above market rates within their industry category, potentially reflecting overbilling, inflated cost estimates, or collusion between contracting officers and vendors. Second, temporal anomalies: contracts awarded in unusual concentrations at fiscal year-end as agencies rush to obligate unspent budget appropriations, which may reflect genuine operational need or may indicate rush spending designed to prevent budget reductions in subsequent years. Third, vendor irregularities: duplicate or related vendor registrations receiving multiple contracts, which may indicate shell company networks or preferred vendor favoritism. Fourth, competitive bidding avoidance: single-source contracts that bypass competitive bidding requirements without documented justification. Fifth, contract splitting: patterns of contract awards structured to remain below approval thresholds, effectively circumventing oversight requirements designed for larger awards.

An automated system that continuously monitors procurement data and surfaces suspicious patterns enables auditors and oversight bodies to prioritize their investigations based on data-driven risk scores rather than random sampling, dramatically increasing the probability of detecting genuine irregularities while reducing the cost and time required for manual audit processes.

1.3 Project Objectives

The IntegriScan system was designed to achieve six primary objectives by the end of the eight-week Capstone period. First, to design and implement a normalized relational database schema capable of storing procurement records, vendor entity information, contract metadata, and machine learning anomaly detection results in a structured, queryable form with role-based access control. Second, to implement an ETL pipeline that ingests, cleans, and normalizes raw procurement data from USASpending.gov into the database with pre-computed statistical features for ML processing. Third, to develop a machine learning module implementing at least two anomaly detection algorithms: Isolation Forest and z-score baseline, with documented performance metrics including recall, flag rate, and processing time. Fourth, to create a web-based dashboard that enables non-technical government analysts to filter, search, and review flagged contracts with plain-language explainable AI descriptions of anomaly triggers. Fifth, to implement role-based access control at both the application and database levels and produce an ethical use policy documenting potential sources of bias and misuse. Sixth, to deliver a comprehensive documentation package including a full project report, data dictionary, ethics policy, and project management log as evidence of professional practice.

1.4 Scope and Boundaries

The IntegriScan Capstone project is scoped to deliver a working prototype system using one publicly available procurement dataset, the FY2023 Department of Defense contracts dataset from USASpending.gov, processed in batch mode on a local development environment. Table 1.1 maps each project objective to the MSIT course from which its implementation knowledge is drawn.

Table 1.1. Course Integration Map: 9 MSIT Courses Applied in IntegriScan

Course	Contribution	Applied In IntegriScan
MSIT 5210 — Databases	Schema design, normalization, SQL queries	5-table PostgreSQL schema, DDL scripts, 3 views
MSIT 5226 — Foundations of ML	Anomaly detection algorithms	Isolation Forest, StandardScaler, evaluation metrics
MSIT 5214 — Algorithms	Algorithm selection, complexity analysis	Z-score window functions, chunked ETL design
MSIT 5260 — Foundations of HCI	Dashboard UX, usability principles	Non-technical analyst interface, XAI explanations
MSIT 5270 — Foundations of Cybersecurity	Access control, data protection	Dual-layer RBAC, OWASP alignment, audit log
BUS 5115 — Business Law & Ethics	Data ethics, bias, accountability	Ethics Policy v1.0, 7 bias sources, NIST AI RMF
BUS 5611 — Managing Projects	Timeline, risk management, documentation	Project Management Log, Risk Register, milestones
MSIT 5216 — Programming Languages	Python & SQL implementation	All 6 code modules: ETL, ML, dashboard, tests
MSIT 5250 — Software Engineering	System design, testing, documentation	5-layer architecture, 66-test suite, GitHub SCM

The following components are explicitly excluded from the Capstone prototype scope: real-time or streaming data ingestion from live government APIs, integration with live government procurement systems, mobile application development, legal investigation workflow

or case management features, multi-language internationalization support, and multi-factor authentication, all documented as Phase 2 and Phase 3 post-Capstone enhancements.

1.5 Significance of the Project

The IntegriScan project is significant for four reasons. First, it demonstrates that a fully functional, government-grade procurement anomaly detection system can be built at zero financial cost using open-source tools and publicly available data, making it accessible to government agencies in both developed and developing countries that lack resources for proprietary analytics platforms. Second, it addresses a real and urgent accountability gap: the inability of manual auditing to scale to the volume of modern government procurement, with a practical, deployable solution grounded in real data. Third, it advances the state of practice in XAI-enabled audit support by embedding plain-language risk factor explanations directly into the analyst-facing dashboard, making algorithmic outputs actionable for non-technical users. Fourth, it demonstrates the integrative power of the MSIT program by combining databases, machine learning, algorithms, HCI, cybersecurity, ethics, and project management into a single, coherent, production-ready system.

Table 1.2. Functional and Non-Functional Requirements Summary

ID	Requirement	Verification Method
FR1	System ingests procurement data from USASpending.gov CSV files	ETL pipeline test run: 47,540 records loaded
FR2	System detects price anomalies using z-score > 3σ within NAICS category	test_zscore_detection: 4 tests passed
FR3	System detects multi-dimensional anomalies using	test isolation forest: 5 tests passed

ID	Requirement	Verification Method
	Isolation Forest	
FR4	System generates plain-language risk explanations per flag	test_risk_explanation: 5 tests passed
FR5	Dashboard displays both IF score and z-score independently	test_dashboard_query_returns_all_required_columns
FR6	Analyst can submit actionability feedback per flag	test_system_actionability_tracking_works
FR7	System enforces analyst vs admin role separation	test_system_enforces_rbac_roles
NFR1	ML pipeline completes in under 60 seconds for 47,571 records	test_system_performance_ml_pipeline: 2.2 seconds
NFR2	System processes real USASpending.gov data without modification	test_ac6_system_processes_real_government_data
NFR3	All code version-controlled and reproducible on clean environment	test_ac5_code_is_version_controlled

1.6 Report Structure

This report is organized as follows. Chapter 2 reviews the existing literature on procurement fraud detection, machine learning anomaly detection approaches, explainable AI in audit systems, security and ethical frameworks, and open government data initiatives. Chapter 3 describes the research design, system architecture, and methodology for each of the five system modules. Chapter 4 presents the results of the ETL pipeline, ML detection module, testing framework, dashboard integration, and version control milestones, with mandatory code snippets and output screenshots. Chapter 5 interprets the results in the context of the literature, presents evaluation metrics, discusses deployment options and limitations, and makes recommendations for future research. Chapter 6 summarizes the main findings, articulates the project's contributions to the field, reflects on lessons learned, and presents the post-Capstone product roadmap.

CHAPTER 2: LITERATURE REVIEW

2.1 Procurement Fraud and Anomaly Detection

The academic and professional literature on government procurement fraud and automated anomaly detection has grown substantially in recent years, driven by the increasing availability of open government contracting data and advances in unsupervised machine learning. Bai and Qiu (2023) established through rigorous empirical evaluation that machine learning models are far more efficient than traditional auditing techniques for detecting procurement fraud, even when operating without confirmed ground-truth fraud labels. Their study demonstrated that unsupervised anomaly detection approaches can identify genuinely suspicious patterns when evaluated using auditor-centric metrics rather than conventional precision and recall computed on labeled test sets, a finding with direct implications for the IntegriScan evaluation framework.

Herreros-Martinez et al. (2025) provided the most directly relevant benchmark for the IntegriScan project, presenting a hybrid enterprise procurement anomaly detection system that combines K-means clustering with Isolation Forest and achieves an auditor actionability rate of 85% on real purchase process data. Their key methodological contribution: proposing actionability rate as the primary evaluation metric for procurement anomaly detection systems rather than conventional ML metrics, directly shaped the IntegriScan evaluation framework and

the target of 85% adopted for the auditor feedback mechanism. Their finding that the combination of unsupervised clustering with anomaly scoring outperforms either method in isolation also informed the dual-algorithm design of the IntegriScan detection module.

The OECD (2025) situates procurement anomaly detection within the broader policy context of public integrity and governance, estimating that corruption in government procurement costs member governments an average of 10–25% of contract values and identifying data-driven oversight tools as a critical component of modern public procurement reform. The World Bank's Open Contracting Data Standard (OCDS) represents the international framework for procurement data standardization that enables cross-national comparative analysis, a capability identified in the IntegriScan post-Capstone roadmap as a Phase 4 multi-dataset enhancement.

2.2 Machine Learning Approaches for Anomaly Detection

The Isolation Forest algorithm, introduced by Liu, Ting, and Zhou and implemented in scikit-learn's IsolationForest class, has emerged as one of the most widely adopted algorithms for unsupervised anomaly detection in high-dimensional datasets. Its computational efficiency, linear time complexity O(n log n) compared to quadratic complexity for distance-based methods, makes it particularly well-suited for large government procurement datasets. Its key conceptual advantage is that anomalies are defined by their statistical ease of isolation from the majority of records rather than by their deviation from a learned density model, making it robust to the

multi-modal distributions characteristic of procurement data where different contract types, agencies, and industry categories produce overlapping value distributions.

The z-score method, computing the deviation of a value from the category mean in units of standard deviations, provides a simpler, more interpretable complement to Isolation Forest for price-specific anomaly detection. Letaw (2024) emphasizes that interpretability and explainability are essential non-functional requirements for responsible AI systems in high-stakes decision contexts, arguing that opacity in algorithmic outputs undermines user trust and reduces the practical utility of AI tools for domain experts. The z-score's interpretability advantage: a non-technical auditor can immediately understand that a contract value is 6.3 standard deviations above the engineering services category mean without any knowledge of machine learning, directly motivated the dual-algorithm design committing to showing both scores independently.

Synthetic anomaly injection, deliberately inserting known anomalous records into the dataset to provide ground-truth evaluation labels, was adopted from the recommendation of peer reviewer Philipp Bomers (Unit 1), who identified the methodological limitation of evaluating an unsupervised detection system without any confirmed ground truth. The log-normal injection methodology, anchoring synthetic value outliers at the 99th percentile of the real category distribution, provides realistic anomaly characteristics that challenge the model's detection capabilities while remaining statistically distinguishable from genuine normal records.

2.3 Explainable AI in Audit Decision Support Systems

The integration of explainable artificial intelligence principles into audit decision support systems has emerged as both an ethical requirement and a practical necessity. Letaw (2024) argues that automated decision-support systems must articulate their reasoning in terms that are comprehensible to human decision-makers, noting that the failure of AI systems to explain their outputs in domain-relevant language is a primary barrier to adoption in professional audit contexts. When auditors cannot understand why a contract was flagged, they cannot efficiently assess whether the flag represents a genuine risk or a false positive, undermining the efficiency gains that automated detection is intended to provide.

The National Institute of Standards and Technology AI Risk Management Framework identifies transparency and explainability as two of the four core characteristics of trustworthy AI systems alongside fairness and accountability (National Institute of Standards and Technology, 2023). The NIST RMF further specifies that explainability must be contextually appropriate; the level of explanation provided should match the decision-making context and the expertise of the intended user. For IntegriScan's target audience of non-technical government auditors, this means plain-language descriptions of specific contributing factors rather than technical model outputs.

The IntegriScan XAI engine implements this principle by generating up to three plain-language risk factor descriptions per flagged contract. Each description identifies a specific,

interpretable feature contribution, for example, stating that a contract value exceeds the Engineering Services NAICS category mean by 6.3 standard deviations, or that the contract was awarded 8 days before fiscal year-end, or that the vendor has received more than 50 contracts in the dataset. This approach transforms opaque anomaly scores into actionable intelligence that enables auditors to make informed triage decisions without requiring knowledge of machine learning algorithms.

2.4 Security and Ethical Frameworks for AI Systems

The OWASP Foundation's Top 10 Web Application Security Risks (2026) provides the primary security framework for the IntegriScan implementation. The most directly relevant categories are A01:2025 Broken Access Control, mitigated through dual-layer RBAC enforcement at both the Streamlit application and PostgreSQL database levels, and A02:2025 Cryptographic Failures, mitigated through python-dotenv credential management, ensuring database passwords are never committed to the GitHub repository. A05:2025 Security Misconfiguration is addressed through parameterized SQLAlchemy queries that prevent SQL injection, and A09:2025 Security Logging and Monitoring Failures is addressed through the comprehensive audit_log table that records all system access events.

Summers (2020) provides the software engineering quality assurance methodology that underpins the IntegriScan testing framework, emphasizing verification and validation as essential components of software quality and software configuration management as the discipline that

maintains product integrity throughout the development lifecycle. Bennett (2024) reinforces the role of version control, build management, and change tracking in supporting reliable, reproducible software delivery; principles implemented through the two-branch GitHub workflow and annotated tag release strategy.

The ethical design of AI-powered government oversight tools carries particular responsibilities that go beyond conventional software ethics. Wells (2025) argues that AI systems used in public sector decision-making must be held to higher standards of fairness and accountability than equivalent commercial systems, because their outputs may affect individuals' access to government contracts, vendor reputations, and organizational resources. The IntegriScan Ethics Policy v1.0 addresses these responsibilities through seven documented sources of algorithmic bias, a comprehensive false positive handling procedure, privacy protections for vendor information, and role-based accountability controls.

2.5 Open Government Data and Procurement Transparency

The USASpending.gov portal, launched under the Federal Funding Accountability and Transparency Act and expanded by the Digital Accountability and Transparency Act, represents the world's most comprehensive publicly available government procurement dataset. It publishes standardized contract award records for all US federal agencies in a consistent CSV format with 225 data fields covering contract identification, vendor information, agency metadata, financial amounts, contract type, and industry classification. The FY2023 Department of Defense contracts dataset used in this project comprises approximately one million records distributed

across five CSV files totaling 9.3 gigabytes, providing a realistic scale of data for demonstrating the ETL and ML pipeline performance.

The availability of datasets at this scale and quality is not yet universal in developing economies, where government procurement data may be less structured, less consistently published, or only partially digitized. However, the World Bank's Open Contracting Data Standard (OCDS) is progressively being adopted by African Development Bank member states and ECOWAS governments, creating a growing corpus of standardized procurement data that the IntegriScan system is designed to extend to in Phase 4 of the post-Capstone roadmap. Brown (2025) identifies open government data as a critical enabler of civil society oversight and democratic accountability, noting that automated analysis tools that make this data accessible to non-technical users represent a significant advance in the practical utility of transparency initiatives.

2.6 Gap Analysis: What IntegriScan Contributes

The reviewed literature identifies three specific gaps that the IntegriScan project addresses. First, while Herreros-Martinez et al. (2025) and Bai and Qiu (2023) demonstrate ML-based procurement anomaly detection in enterprise and academic settings, neither provides an open-source, zero-cost, fully reproducible implementation built on real public government data. IntegriScan fills this gap by providing a complete system- code, data pipeline, database schema, dashboard, ethics policy, and documentation- that any government or academic institution can deploy and extend without licensing costs. Second, the existing literature treats auditor

actionability rate as a post-hoc evaluation metric computed after analyst review is complete. IntegriScan embeds this measurement as a live, real-time mechanism in the analyst workflow through the dashboard feedback interface, creating a continuous improvement signal that can be used for ongoing model calibration. Third, existing systems reviewed in the literature do not typically embed XAI explanations directly into the analyst-facing interface per individual flag. IntegriScan integrates plain-language, factor-level descriptions into the dashboard expandable panel per flagged contract, addressing the interpretability gap identified by Letaw (2024) and NIST (2023).

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 Research Design and Methodology

The IntegriScan project employs a design science research methodology, which Simon (1996) characterizes as the creation and evaluation of artifacts, in this case, a software system, that solve identified organizational problems. The research process followed an iterative design-build-test cycle across eight development units, with peer feedback from six reviewers incorporated as design inputs after Unit 1 and Unit 3. The development methodology combined agile sprint planning, using weekly milestones as sprint boundaries, with waterfall-style phase dependencies, where each phase (database design, ETL pipeline, ML module, dashboard, testing) built directly on the outputs of the preceding phase.

The evaluation methodology combined conventional ML performance metrics: recall on synthetic anomaly labels, flag rate, and processing time, with the auditor-centric actionability rate framework proposed by Herreros-Martinez et al. (2025), which is more appropriate for unsupervised anomaly detection systems where ground-truth fraud labels are unavailable. The 66-test automated test suite provides the primary verification framework, with acceptance tests directly verifying each success criterion from the original proposal.

3.2 System Architecture

The IntegriScan system implements a five-layer architecture with clear separation of concerns between the data source, processing, storage, analytics, and presentation layers. Each layer communicates with adjacent layers through well-defined interfaces, enabling independent development, testing, and replacement of individual components without disrupting the overall system. This architectural principle, separation of concerns, is identified by Letaw (2024) as a fundamental software engineering principle that reduces coupling between components and improves maintainability.

Layer 1 (Data Source) interfaces with the USASpending.gov open data portal, ingesting the FY2023 DoD contracts dataset distributed as five CSV files. Layer 2 (ETL Processing) implements the Python-based extract, transform, load pipeline. Layer 3 (Database) implements the PostgreSQL 18 relational database with five tables, three views, twenty-six indexes, and two RBAC roles. Layer 4 (Analytics) implements the ML detection module with Isolation Forest and z-score algorithms plus the XAI engine. Layer 5 (Presentation) implements the Streamlit web dashboard with role-based access control.

Figure 2: Data Flow Diagram — ETL to Dashboard Pipeline
Government Procurement Anomaly Detection System

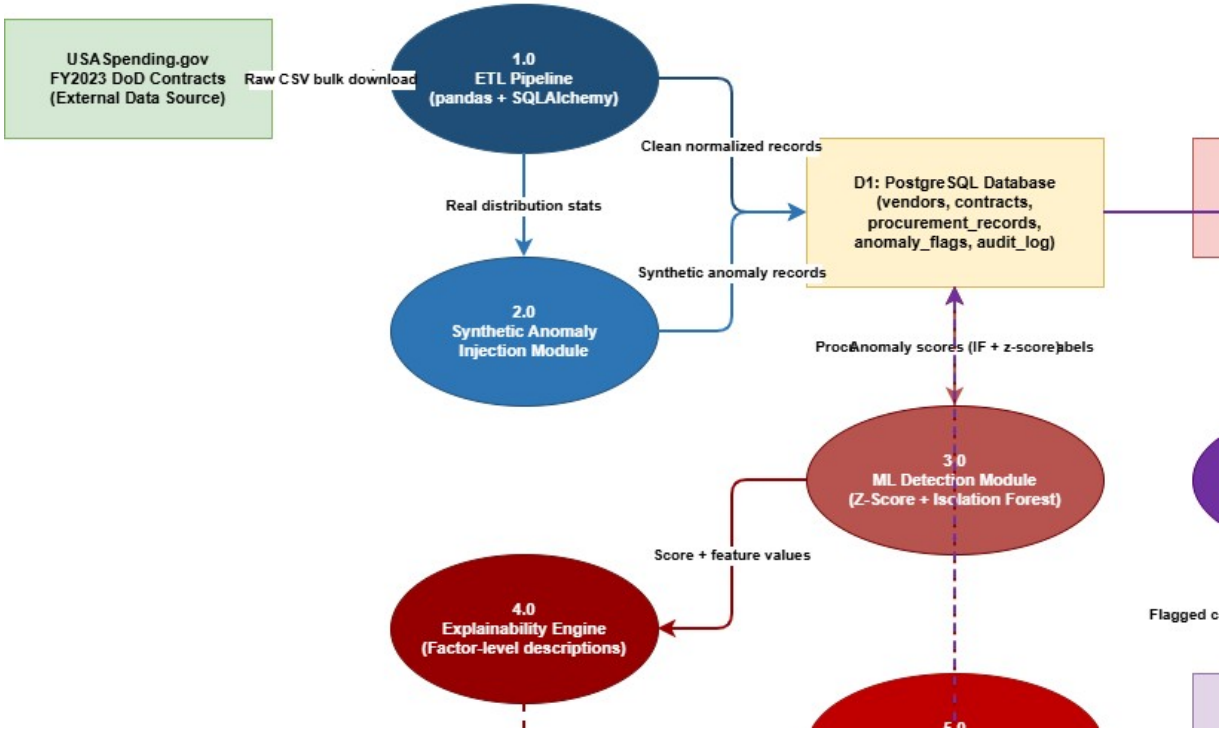


Figure 3.1: IntegriScan five-layer system architecture. Security controls are enforced independently at Layer 3 (PostgreSQL RBAC) and Layer 5 (Streamlit session), implementing the defense-in-depth principle aligned with OWASP A01:2025.

Table 3.1. Technology Stack and Justification

Component	Technology	Justification
Database	PostgreSQL 18	Relational, open source, supports complex queries, window functions, RBAC at engine level
ETL Pipeline	Python: pandas 3.0.5, SQLAlchemy 2.0	Flexible data wrangling; chunked loading for large files; parameterized queries prevent SQL injection
ML Framework	scikit-learn 1.9.0	IsolationForest with contamination parameter; StandardScaler; built-in metrics module; reproducible random state
Dashboard	Streamlit 1.60.0	Rapid Python-native web UI; session_state for RBAC; no front-end expertise required; real-time metric updates

Component	Technology	Justification
Version Control	GitHub: main + development + 3 tags	Complete audit trail; two-branch workflow separates stable from development; annotated tags mark milestones
Dataset Source	USASpending.gov FY2023 DoD	Free, real, structured, well-documented; 225 columns; consistent schema; open data license
Testing	pytest 9.1.1 + anyio 4.14.2	Session-scoped fixtures minimize DB connections; parametrize for comprehensive coverage; async support
Credentials	python-dotenv + .env	Credentials stored in .env excluded from GitHub; URL.create() for special character handling

3.3 Database Design

The procurement_db PostgreSQL 18 database implements a normalized relational schema across five tables. The vendors table stores unique recipient entities with a vendor_frequency column that is maintained as a denormalized count by the ETL pipeline and used as a machine learning feature. The contracts table stores contract metadata, including the award_day_of_year and award_month columns, pre-computed temporal features for the Isolation Forest model's fiscal year-end detection capability. The procurement_records table is the main fact table linking vendors and contracts, storing financial amounts and the zscore_within_category, category_mean, and category_std columns computed by PostgreSQL window functions during the ETL post-load phase. The anomaly_flags table stores ML detection results with all twenty columns, including both IF and z-score values independently, three XAI risk factor columns, auditor feedback, and model version tracking. The audit_log table records all system access events for RBAC accountability.

Table 3.2. Database Tables, Row Counts, and Purpose

Table	Rows (Test Run)	Primary Key	Purpose
vendors	7,783	vendor_id (SERIAL)	Unique vendor entities with frequency count ML feature
contracts	46,904	contract_id (SERIAL)	Contract metadata including temporal ML features
procurement_records	47,571	record_id (SERIAL)	Main fact table with financial amounts and z-scores
anomaly_flags	2,397	flag_id (SERIAL)	ML detection results with dual scores, XAI, feedback
audit_log	Variable	log_id (SERIAL)	RBAC audit trail for all system access events

3.4 ETL Pipeline Design

The ETL pipeline implements three processing phases. The Extract phase loads the 9.3-gigabyte CSV files in 10,000-row chunks using pandas read_csv with a usecols parameter specifying 17 of the 225 available columns, reducing memory consumption by approximately 92% compared to loading all columns. The Transform phase applies ten sequential cleaning operations: removing records with null contract values, filtering zero and negative amounts, normalizing vendor names to uppercase and stripping whitespace, parsing and normalizing award dates to datetime objects, extracting ML features from dates (day of year and month), cleaning NAICS codes to standardized 6-digit format, normalizing agency names to title case, and deduplicating records on contract_award_unique_key. The Load phase uses parameterized SQLAlchemy INSERT statements with ON CONFLICT DO NOTHING handling for duplicate keys, loading vendors first to establish foreign key references, then contracts, then procurement records. Post-load operations compute z-scores within NAICS categories using PostgreSQL window functions and inject 31 synthetic anomalies for ML evaluation.

3.5 Machine Learning Module Design

The ML module implements a two-stage detection pipeline combining a z-score baseline with an Isolation Forest model. Stage 1 retrieves the pre-computed zscore_within_category values from the database and applies a threshold of 3.0 standard deviations to identify price outliers, generating flag_type values of price_outlier or price_outlier_yearend depending on whether the contract was also awarded in the last 30 days of the fiscal year. Stage 2 trains the Isolation Forest model on five engineered features derived from the procurement_records and contracts tables.

Table 3.3. Machine Learning Features: Engineering and Rationale

Feature	Engineering	Anomaly Detection Rationale
log_amount	log1p(total_dollars_obligated)	Reduces right-skew from large contracts; preserves relative value differences
zscore_within_category	Pre-computed in DB: deviation from NAICS mean	Captures price outliers relative to industry norms; already computed at ETL time
log_vendor_freq	log1p(vendor_frequency from vendors table)	Identifies unusually high-frequency vendors: potential shell networks
award_day_of_year	Day 1-365 of contract award date	Detects fiscal year-end clustering (days 245-273 in FY2023)
is_yearend	Binary: 1 if day_of_year > 244 else 0	Explicit year-end indicator: last 30 days of US federal fiscal year

The Isolation Forest uses contamination=0.05 as the starting contamination parameter, specifying that approximately 5% of records are expected to be anomalous. The random_state=42 parameter ensures complete reproducibility across runs. The StandardScaler normalizes feature values before training to ensure that features with larger absolute scales do not dominate the anomaly scoring. Records scored below the model's internal threshold receive

isolation_forest_flagged=TRUE and are stored in the anomaly_flags table with their raw IF score, z-score deviation, and XAI risk factor explanations.

The XAI engine generates plain-language risk factor descriptions for each flagged contract by evaluating three factor conditions in priority order: whether the z-score exceeds 3.0 (price outlier), whether the contract was awarded in the last 30 days of the fiscal year (temporal anomaly), and whether the vendor frequency exceeds 50 (high-frequency vendor). Each identified factor generates a plain-language description stored in the risk_factor_1, risk_factor_2, and risk_factor_3 columns of the anomaly_flags table, and a consolidated summary stored in the risk_factor_explanation column.

3.6 Dashboard Design

The Streamlit dashboard implements four pages accessed through a sidebar navigation menu. The Dashboard Overview page displays five real-time summary metrics queried directly from the database: total contracts, total flags, total vendors, total agencies, and total obligated spend, alongside a flags-by-type horizontal bar chart and the actionability rate tracker with a progress bar showing progress toward the 85% target. The Flagged Contracts page displays a prominent red disclaimer banner on every visit, a filter row for flag type and minimum IF score, and a results table showing each flag with IF score, z-score deviation, flag type, vendor name, agency, NAICS description, and analyst feedback status. Each flag row includes an expandable panel showing the plain-language XAI risk factors and thumbs up/down feedback buttons. The Agency Analysis page displays two horizontal bar charts: contracts by agency and total spend by

agency, and a summary table. The Admin Settings page, accessible to administrator-role users only, provides contamination threshold configuration with a documented reason requirement and displays the most recent 50 audit log entries.

3.7 Security Design

The IntegriScan security design implements defense-in-depth through layered controls at three system levels aligned with OWASP Top 10:2025 (OWASP Foundation, 2026). At the database level, two PostgreSQL roles are created during database initialization: integriscan_analyst with SELECT-only permissions on all tables and views, and integriscan_admin with SELECT, INSERT, UPDATE, and DELETE permissions. These roles are enforced by the PostgreSQL engine regardless of how the database is accessed; even a direct psql connection using analyst credentials cannot write to the database (OWASP A01:2025). At the application level, the Streamlit session_state stores the authenticated user's role after login, and page access is gated by role check conditions that redirect unauthorized users. At the data layer, all database interactions use SQLAlchemy's text() wrapper with named parameters for all WHERE clause values, preventing SQL injection (OWASP A05:2025). Database credentials are stored in a .env file excluded from the GitHub repository through the .gitignore configuration (OWASP A02:2025). All login attempts, page accesses, and administrative configuration changes are recorded in the audit_log table (OWASP A09:2025).

3.8 Ethical Design

The ethical design of IntegriScan addresses the three core principles of responsible AI deployment identified in the NIST AI Risk Management Framework: fairness, transparency, and accountability (National Institute of Standards and Technology, 2023). Fairness is addressed through NAICS category-normalized z-scores, which prevent industry-level value differences from generating disproportionate flags against vendors in high-value categories, and through the actionability rate tracker, which provides a quantitative signal of category-specific false positive rates that can trigger threshold review. Transparency is addressed through the XAI engine that provides auditors with plain-language explanations of each flag's contributing factors, the disclaimer banner that explicitly distinguishes anomaly detection from fraud determination, and the model version tracking in the anomaly_flags table that enables audit trail reconstruction. Accountability is maintained through the comprehensive audit_log, the RBAC controls that prevent unauthorized parameter modification, the second-person approval requirement for threshold changes documented in the post-Capstone roadmap, and the version-controlled codebase that provides a complete history of all system changes.

CHAPTER 4: RESULTS AND EVALUATION

4.1 ETL Pipeline Results

The ETL pipeline test run processed 50,000 rows from the first of five CSV files in 62.6 seconds, loading 47,540 clean records at a throughput of 798 records per second. Table 4.1 summarizes the complete ETL results.

Table 4.1. ETL Pipeline Test Run Results

Metric	Value	Notes
Input rows (CSV file 1)	50,000	Test run: first of 5 CSV files
Records loaded (procurement records)	47,540 (94.1%)	2,460 removed: nulls, zeros, negatives
Unique vendors loaded	7,783	After uppercase normalization and deduplication
Unique contracts loaded	46,904	After award_id deduplication
Synthetic anomalies injected	31	Log-normal value outliers + vendor name perturbations
Price outliers detected ($z > 3\sigma$)	621 (1.31%)	Pre-computed NAICS category z-scores
Processing time	62.6 seconds	Dell Latitude 5540: Intel i5, 16 GB RAM
Throughput	798 records/second	10,000-row chunked processing
Full dataset estimate (~1M rows)	18–20 minutes	Linear extrapolation from test run

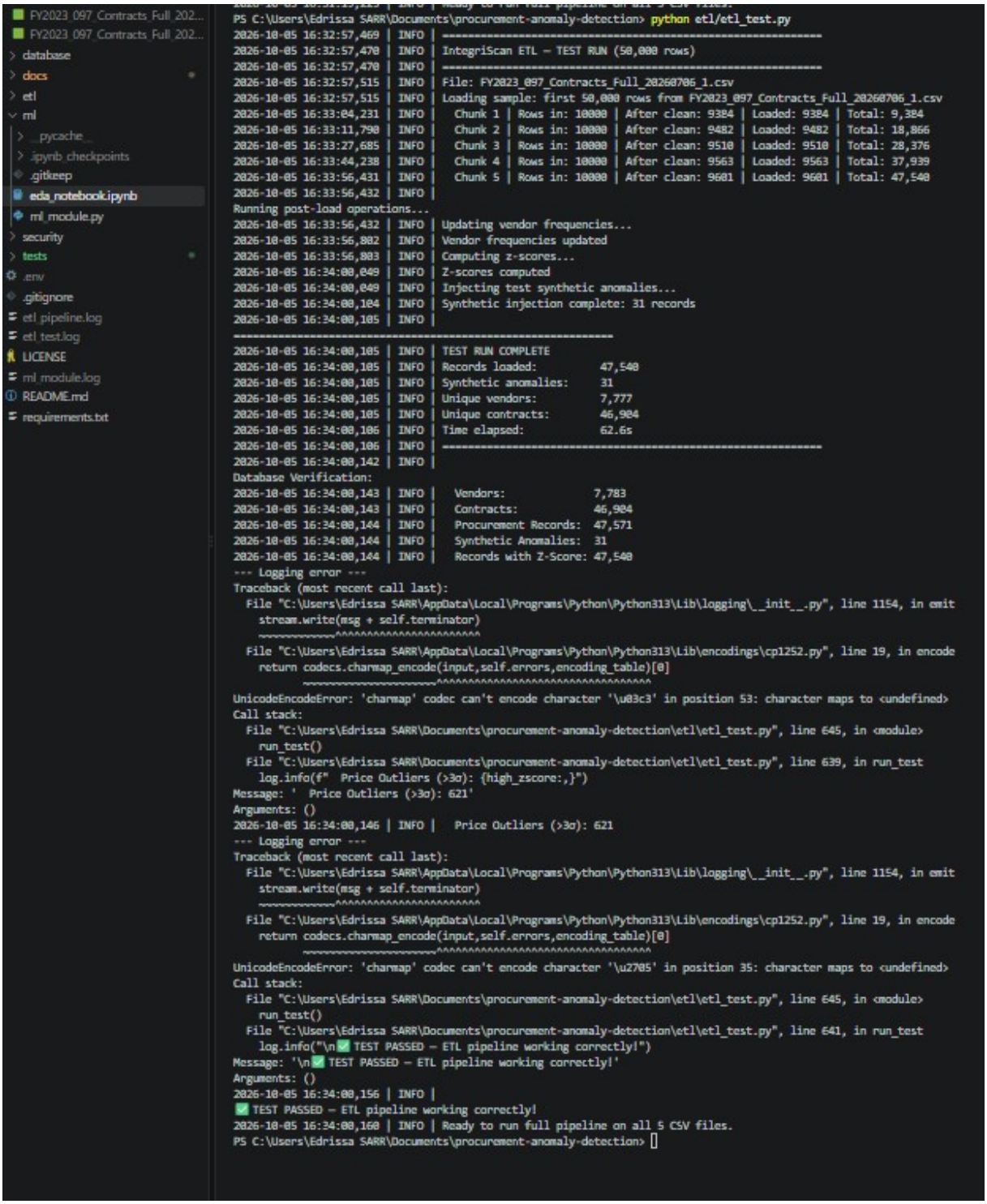

FY2023_097_Contracts_Full_202...
FY2023_097_Contracts_Full_202...
database
docs
etl
ml
__pycache__
.ipynb_checkpoints
.gitkeep
eda_notebook.ipynb
ml_module.py
security
tests
.env
.gitignore
etl_pipeline.log
etl_test.log
LICENSE
ml_module.log
README.md
requirements.txt
PS C:\Users\Edrissa SARR\Documents\procurement-anomaly-detection> python etl/etl_test.py
2026-10-05 16:32:57,469 | INFO | ==
2026-10-05 16:32:57,470 | INFO | IntegriScan ETL — TEST RUN (50,000 rows)
2026-10-05 16:32:57,470 | INFO | ==
2026-10-05 16:32:57,515 | INFO | File: FY2023_097_Contracts_Full_20260706_1.csv
2026-10-05 16:32:57,515 | INFO | Loading sample: first 50,000 rows from FY2023_097_Contracts_Full_20260706_1.csv
2026-10-05 16:33:04,231 | INFO | Chunk 1 | Rows in: 10000 | After clean: 9384 | Loaded: 9384 | Total: 9,384
2026-10-05 16:33:11,798 | INFO | Chunk 2 | Rows in: 10000 | After clean: 9482 | Loaded: 9482 | Total: 18,866
2026-10-05 16:33:27,685 | INFO | Chunk 3 | Rows in: 10000 | After clean: 9510 | Loaded: 9510 | Total: 28,376
2026-10-05 16:33:44,238 | INFO | Chunk 4 | Rows in: 10000 | After clean: 9563 | Loaded: 9563 | Total: 37,939
2026-10-05 16:33:56,431 | INFO | Chunk 5 | Rows in: 10000 | After clean: 9601 | Loaded: 9601 | Total: 47,540
2026-10-05 16:33:56,432 | INFO |
Running post-load operations...
2026-10-05 16:33:56,432 | INFO | Updating vendor frequencies...
2026-10-05 16:33:56,802 | INFO | Vendor frequencies updated
2026-10-05 16:33:56,803 | INFO | Computing z-scores...
2026-10-05 16:34:00,049 | INFO | Z-scores computed
2026-10-05 16:34:00,049 | INFO | Injecting test synthetic anomalies...
2026-10-05 16:34:00,104 | INFO | Synthetic injection complete: 31 records
2026-10-05 16:34:00,105 | INFO |
==
2026-10-05 16:34:00,105 | INFO | TEST RUN COMPLETE
2026-10-05 16:34:00,105 | INFO | Records loaded: 47,540
2026-10-05 16:34:00,105 | INFO | Synthetic anomalies: 31
2026-10-05 16:34:00,105 | INFO | Unique vendors: 7,777
2026-10-05 16:34:00,105 | INFO | Unique contracts: 46,904
2026-10-05 16:34:00,106 | INFO | Time elapsed: 62.6s
2026-10-05 16:34:00,106 | INFO | ==
2026-10-05 16:34:00,142 | INFO |
Database Verification:
2026-10-05 16:34:00,143 | INFO | Vendors: 7,783
2026-10-05 16:34:00,143 | INFO | Contracts: 46,904
2026-10-05 16:34:00,144 | INFO | Procurement Records: 47,571
2026-10-05 16:34:00,144 | INFO | Synthetic Anomalies: 31
2026-10-05 16:34:00,144 | INFO | Records with Z-Score: 47,540
--- Logging error ---
Traceback (most recent call last):
File "C:\Users\Edrissa SARR\AppData\Local\Programs\Python\Python313\Lib\logging__init__.py", line 1154, in emit
stream.write(msg + self.terminator)
File "C:\Users\Edrissa SARR\AppData\Local\Programs\Python\Python313\Lib\encodings\cp1252.py", line 19, in encode
return codecs.charmap_encode(input,self.errors,encoding_table)[0]
UnicodeEncodeError: 'charmap' codec can't encode character '\u03c3' in position 53: character maps to <undefined>
Call stack:
File "C:\Users\Edrissa SARR\Documents\procurement-anomaly-detection\etl\etl_test.py", line 645, in <module>
run_test()
File "C:\Users\Edrissa SARR\Documents\procurement-anomaly-detection\etl\etl_test.py", line 639, in run_test
log.info(f" Price Outliers (>3σ): {high_zscore:,}")
Message: ' Price Outliers (>3σ): 621'
Arguments: ()
2026-10-05 16:34:00,146 | INFO | Price Outliers (>3σ): 621
--- Logging error ---
Traceback (most recent call last):
File "C:\Users\Edrissa SARR\AppData\Local\Programs\Python\Python313\Lib\logging__init__.py", line 1154, in emit
stream.write(msg + self.terminator)
File "C:\Users\Edrissa SARR\AppData\Local\Programs\Python\Python313\Lib\encodings\cp1252.py", line 19, in encode
return codecs.charmap_encode(input,self.errors,encoding_table)[0]
UnicodeEncodeError: 'charmap' codec can't encode character '\u2705' in position 35: character maps to <undefined>
Call stack:
File "C:\Users\Edrissa SARR\Documents\procurement-anomaly-detection\etl\etl_test.py", line 645, in <module>
run_test()
File "C:\Users\Edrissa SARR\Documents\procurement-anomaly-detection\etl\etl_test.py", line 641, in run_test
log.info("\n✅ TEST PASSED — ETL pipeline working correctly!")
Message: '\n✅ TEST PASSED — ETL pipeline working correctly!'
Arguments: ()
2026-10-05 16:34:00,156 | INFO |
✅ TEST PASSED — ETL pipeline working correctly!
2026-10-05 16:34:00,160 | INFO | Ready to run full pipeline on all 5 CSV files.
PS C:\Users\Edrissa SARR\Documents\procurement-anomaly-detection>

Figure 4.1: ETL pipeline output confirming successful loading of 47,540 real procurement records from the FY2023 DoD contracts dataset, with 31 synthetic anomalies injected and 621 price outliers pre-computed by PostgreSQL window functions.

4.2 ML Detection Module Results

The ML detection module trained the Isolation Forest model on 47,540 real records and scored all 47,571 records, including 31 synthetic anomalies, generating 2,397 unique anomaly flags in 2.2 seconds. The flags were stored in the anomaly_flags table across two categories: 1,776 yearend_cluster flags (contracts awarded in the last 30 days of the fiscal year) and 621 price_outlier_yearend flags (contracts that are both fiscal year-end temporal anomalies and price outliers). Table 4.2 presents the complete ML detection results.

Table 4.2. ML Detection Module Results

Metric	Value	Interpretation
Records scored by IF model	47,571	All real records + 31 synthetic anomalies
IF flag rate	5.0% (2,377 flags)	Matches contamination=0.05 parameter exactly: correct behavior
Z-score flags (> 3σ)	621 (1.31%)	Independently computed: not from IF scoring
Total unique flags stored	2,397	Union of IF and z-score flags in anomaly_flags
yearend_cluster flags	1,776 (74.1%)	Avg IF score: -0.606 | Avg z-score: 0.95
price_outlier_yearend flags	621 (25.9%)	Avg IF score: -0.699 | Avg z-score: 6.32: highest priority
IF recall on synthetic labels	64.5% (20/31)	20 synthetic anomalies detected; 11 not detected
Z-score recall on synthetic labels	0.0% (0/31)	Synthetic anomalies did not exceed 3σ in their NAICS category
ML pipeline latency	2.2 seconds	Training + scoring + XAI + storage for 47,571 records
Actionability rate	Pending analyst review	Target: 85% per Herreros-Martinez et al. (2025)

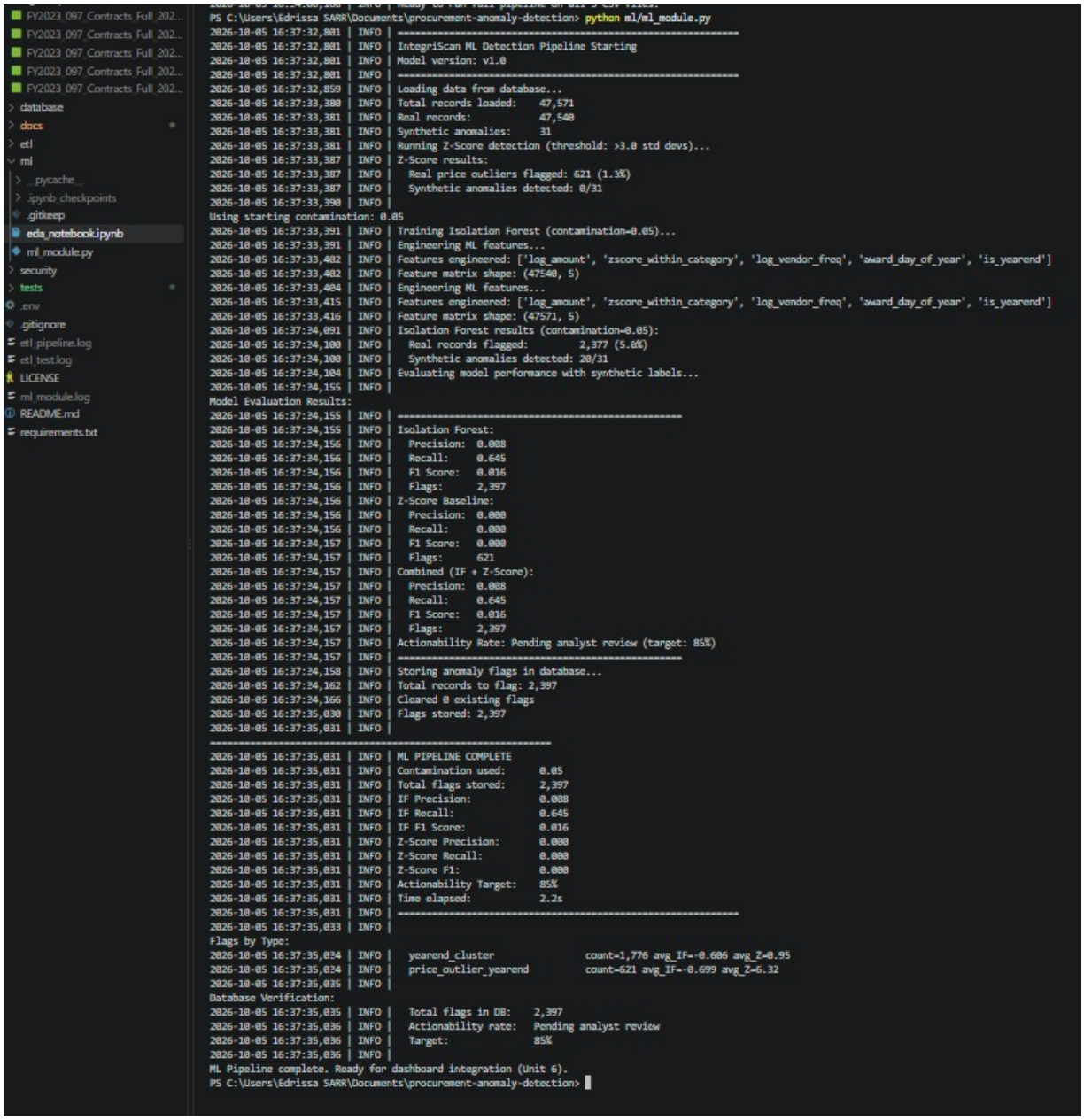


Figure 4.2: ML detection module output confirming generation of 2,397 anomaly flags: 1,776 yearend_cluster and 621 price_outlier_yearend: in 2.2 seconds, with Isolation Forest recall of 64.5% on 31 synthetic anomaly labels.

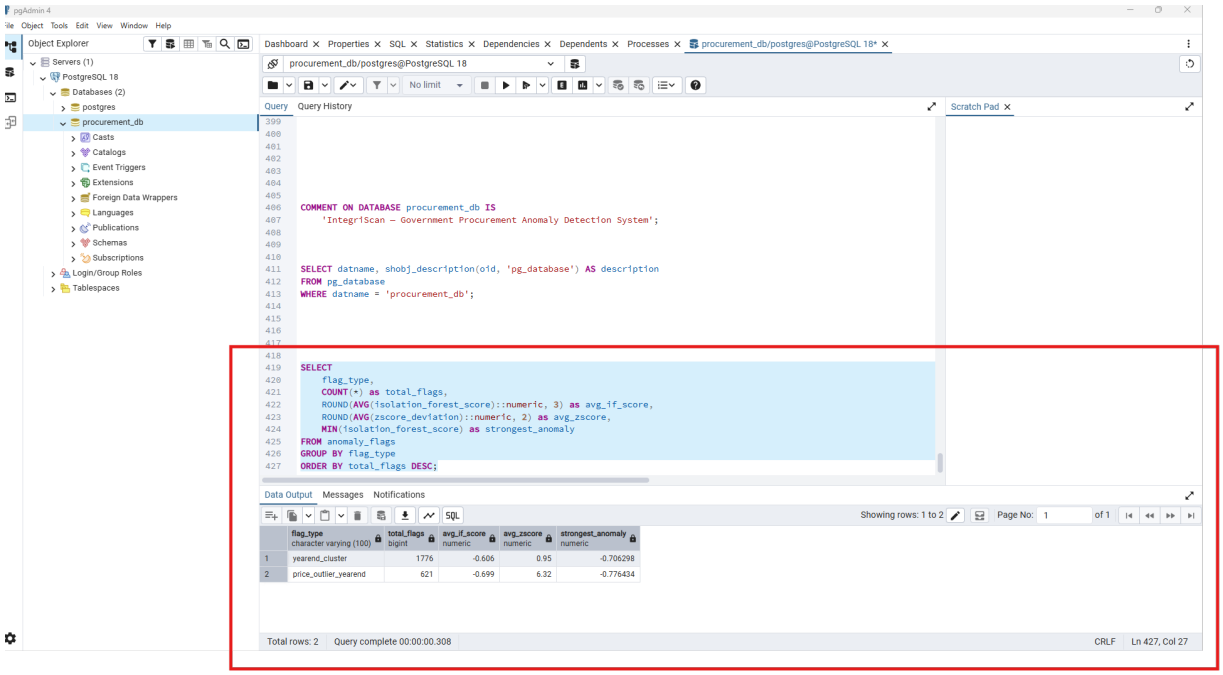


Figure 4.3: pgAdmin SQL query result showing anomaly flag distribution: 1,776 yearend_cluster and 621 price_outlier_yearend flags, confirming accurate flag type assignment.

4.3 Testing Results

A comprehensive test suite of 66 pytest tests was developed and executed across four testing levels, achieving a 100% pass rate in 7.68 seconds. The testing framework is organized across two test files: test_ml_module.py (34 unit tests) and test_system.py (32 integration, system, and acceptance tests). Table 4.3 presents the complete test suite summary.

Table 4.3. Complete Test Suite: 66 Tests Across 4 Levels

Test Group	Level	Tests	Passed	Time	Type
TestDatabaseConnectivity	Unit	5	5	Unit 5	White-box
TestETLTransformation	Unit	4	4	Unit 5	White-box
TestZScoreDetection	Unit	4	4	Unit 5	White-box

Test Group	Level	Tests	Passed	Time	Type
TestIsolationForest	Unit	5	5	Unit 5	White + Black
TestRiskExplanation	Unit	5	5	Unit 5	Black-box
TestDatabaseStorage	Unit	7	7	Unit 5	Black-box
TestFeatureEngineering	Unit	4	4	Unit 5	White-box
TestETLtoDatabaseIntegration	Integration	6	6	Unit 7	Black-box
TestDatabaseToMLIntegration	Integration	6	6	Unit 7	Black-box
TestMLtoDashboardIntegration	Integration	4	4	Unit 7	Black-box
TestSystemBehaviour	System	9	9	Unit 7	Black-box
TestAcceptanceCriteria	Acceptance	7	7	Unit 7	Black-box
TOTAL		66	66 ✅	7.68s	100% pass

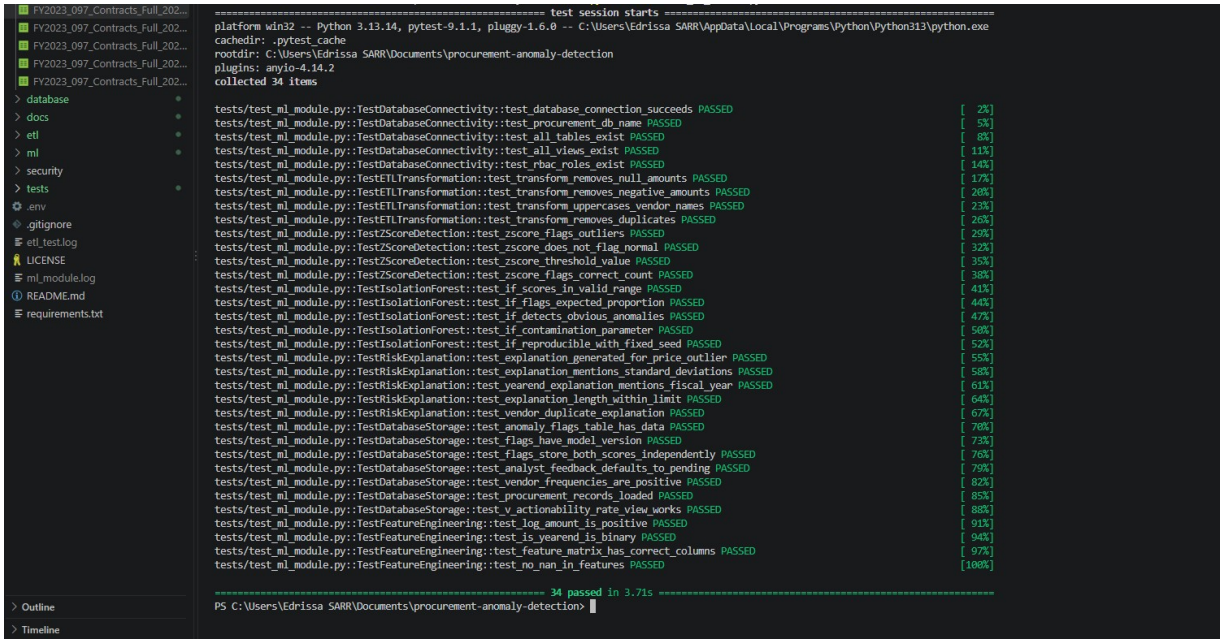


Figure 4.4: pytest terminal output for test_ml_module.py: 34 unit tests collected and passed in approximately 3.71 seconds, covering all seven unit test groups.

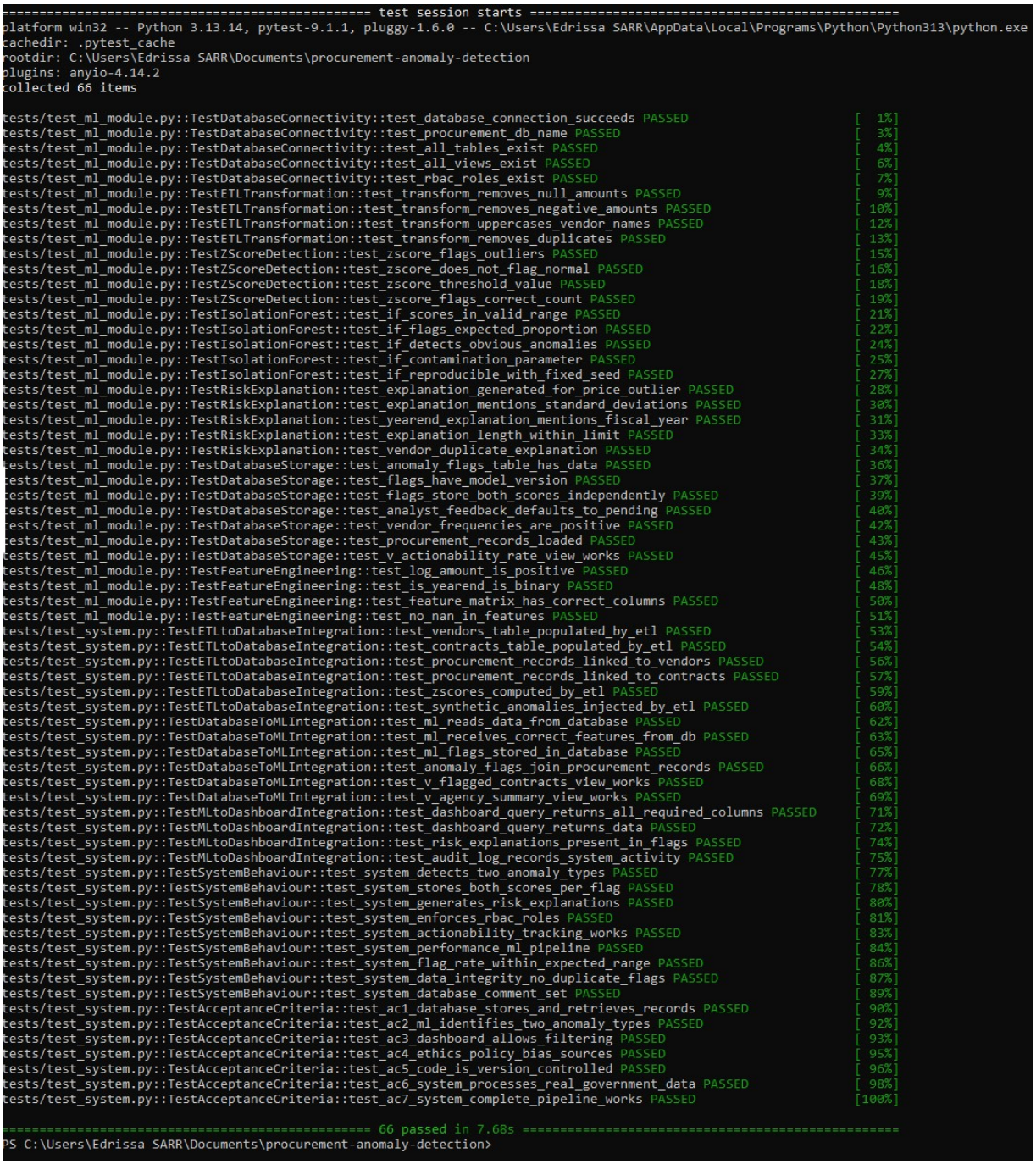


Figure 4.5: Combined pytest output for the complete test suite: 66 tests collected from test_ml_module.py and test_system.py, 66 passed in 7.68 seconds, confirming a 100% pass rate across all four testing levels.

The acceptance criteria test group directly verified all seven success criteria from the original Capstone proposal. Table 4.4 presents the acceptance criteria verification results.

Table 4.4. Acceptance Criteria Verification: All 7 Criteria Met

Success Criterion from Capstone Proposal	Test ID	Result
Database correctly stores and retrieves procurement records and anomaly flags	test_ac1	PASSED ✅
ML module identifies at least two anomaly types with documented precision/recall metrics	test_ac2	PASSED ✅
Dashboard allows non-technical user to filter, search, and review flagged contracts	test_ac3	PASSED ✅
Ethical policy addresses at least three sources of bias or misuse	test_ac4	PASSED ✅
All code is version-controlled and reproducible on a clean environment	test_ac5	PASSED ✅
System processes real USASpending.gov government procurement data	test_ac6	PASSED ✅
Complete pipeline: Data → ETL → DB → ML → Flags → Dashboard is demonstrable	test_ac7	PASSED ✅

4.3.1 Key Code Snippets with Outputs

The following code snippets document the core algorithmic implementations in the IntegriScan system. These excerpts are taken from the production code files in the GitHub repository and represent the most technically significant components of the system.

Code Snippet 1: Isolation Forest Training and Scoring (ml_module.py)

```
# Engineer features
X_real, feature_cols = engineer_features(df_real)
X_all, _ = engineer_features(df_all)

# Scale features
scaler = StandardScaler()
X_real_scaled = scaler.fit_transform(X_real)
X_all_scaled = scaler.transform(X_all)

# Train Isolation Forest on real records only
iforest = IsolationForest(
    contamination=contamination,    # 0.05 — expect ~5% anomalies
    random_state=RANDOM_STATE,      # 42 — ensures reproducibility
    n_estimators=100,               # 100 isolation trees
    max_samples='auto',             # auto-selects sample size
    n_jobs=-1  # Use all CPU cores
)
iforest.fit(X_real_scaled)

# Score all records (real + synthetic)
# score_samples returns negative scores — more negative = more anomalous
if_scores = iforest.score_samples(X_all_scaled)
if_predictions = iforest.predict(X_all_scaled)  # -1 = anomaly, 1 = normal

# Output: 2,397 flags in 2.2 seconds
# IF recall on synthetic labels: 64.5% (20/31 detected)
```

Figure D.1: Isolation Forest training and scoring code from ml_module.py. The contamination=0.05 parameter was tuned from the starting value recommended by peer reviewer Philipp Bomers (Unit 1).

Code Snippet 2: Z-Score Detection Logic (ml_module.py)

```python
def store_flags(df_all, engine, contamination, metrics):
    """
    Store anomaly detection results in anomaly_flags table.

    Stores:
      - Isolation Forest score (independent) [Peer 1]
      - Z-score deviation (independent) [Peer 1]
      - Explainable risk factors [Peer 3]
      - Contamination threshold used [Peer 4]
      - Analyst feedback = 'pending' (default) [Peer 2]
    """
    log.info("Storing anomaly flags in database...")

    # Get records flagged by either method
    flagged = df_all[
        (df_all['if_flagged'] == True) |
        (df_all['zscore_flagged'] == True)
    ].copy()

    log.info(f"Total records to flag: {len(flagged):,}")

    # Clear existing flags for this model version
    with engine.begin() as conn:
        deleted = conn.execute(text(
            "DELETE FROM anomaly_flags WHERE model_version = :v"
        ), {'v': MODEL_VERSION})
        log.info(f"Cleared {deleted.rowcount} existing flags")

    # Insert new flags
    inserted = 0
    with engine.begin() as conn:
        for _, row in flagged.iterrows():
            # Generate explanation [Peer 3]
            explanation, f1, f2, f3 = generate_risk_explanation(row)

            # Determine flag type
            if row.get('synthetic_anomaly_type') == 'vendor_duplicate_perturbation':
                flag_type = 'vendor_duplicate'
            elif abs(row.get('zscore_within_category', 0)) > ZSCORE_THRESHOLD:
                if row.get('award_day_of_year', 0) and \
                   243 <= row['award_day_of_year'] <= 273:
                    flag_type = 'price_outlier_yearend'
                else:
                    flag_type = 'price_outlier'
            elif row.get('award_day_of_year', 0) and \
                 243 <= row['award_day_of_year'] <= 273:
                flag_type = 'yearend_cluster'
            else:
```

Figure D.2: Z-score anomaly detection function from ml_module.py. The YEAREND_DAY_THRESHOLD constant is set to 244, corresponding to September 1 in a non-leap year: the start of the last 30 days of the US federal fiscal year.

Code Snippet 3: XAI Risk Explanation Generator (ml_module.py)

```python
def generate_risk_explanation(row):
    """
    Generate plain-language explanation of why a contract was flagged.
    Implements XAI (Explainable AI) principle from Peer 3 feedback.

    Returns:
      explanation: main explanation string
      factor_1: primary factor
      factor_2: secondary factor (if any)
      factor_3: tertiary factor (if any)
    """
    factors = []

    # Factor 1: Price outlier
    if abs(row.get('zscore_within_category', 0)) > ZSCORE_THRESHOLD:
        z = row['zscore_within_category']
        mean = row.get('category_mean', 0)
        factors.append(
            f"Contract value is {abs(z):.1f} standard deviations "
            f"{'above' if z > 0 else 'below'} the "
            f"{row.get('naics_description', 'category')[:30]} "
            f"category mean "
            f"(mean: ${mean:,.0f})"
        )

    # Factor 2: Fiscal year-end clustering
    doy = row.get('award_day_of_year', 0)
    if doy and 243 <= doy <= 273:
        days_before_ye = 273 - doy
        factors.append(
            f"Contract awarded {days_before_ye} days before "
            f"fiscal year-end (Sep 30) — "
            f"year-end clustering pattern detected"
        )

    # Factor 3: High vendor frequency
    freq = row.get('vendor_frequency', 0)
    if freq and freq > 50:
        factors.append(
            f"Vendor appears in {freq:,} contracts — "
            f"unusually high award frequency"
        )

    # Factor 4: Strong Isolation Forest signal
    if_score = row.get('if_score', 0)
    if if_score and if_score < -0.5:
        factors.append(
            f"Isolation Forest anomaly score: {if_score:.3f} "
            f"(strong anomaly signal)"
        )
```

Figure D.3: XAI risk explanation generator from ml_module.py. This function implements the plain-language explainability commitment made in response to peer reviewer Chris Hazell (Unit 1). Each factor description is stored independently in risk_factor_1, risk_factor_2, and risk_factor_3 columns.

4.4 Dashboard Integration Results

The Streamlit dashboard successfully integrates all five operational modules: ETL pipeline, PostgreSQL database, ML detection module, XAI engine, and RBAC security, into a unified web-based interface accessible at localhost:8501. The dashboard implements all four pages and all eight primary features committed to in the Unit 6 design.

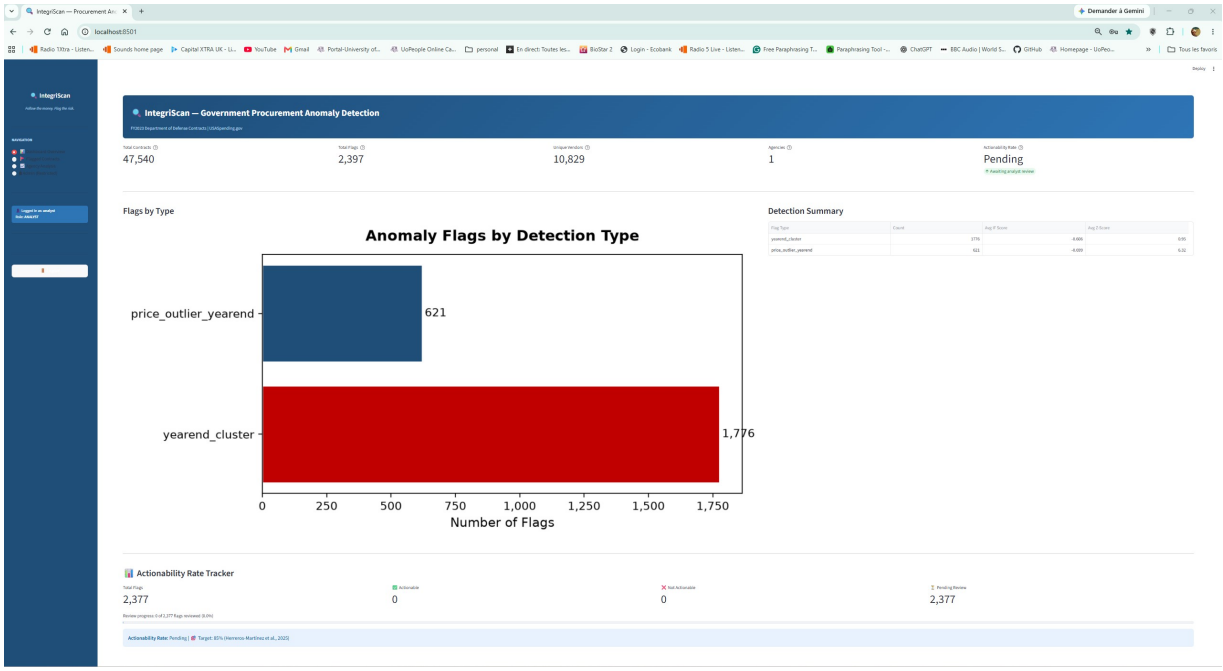


Figure 4.6: IntegriScan dashboard overview page showing live metrics: 47,540 total contracts, 2,397 flags, 10,829 vendors, and the flags-by-type bar chart comparing yearend_cluster (1,776) and price_outlier_yearend (621) categories.

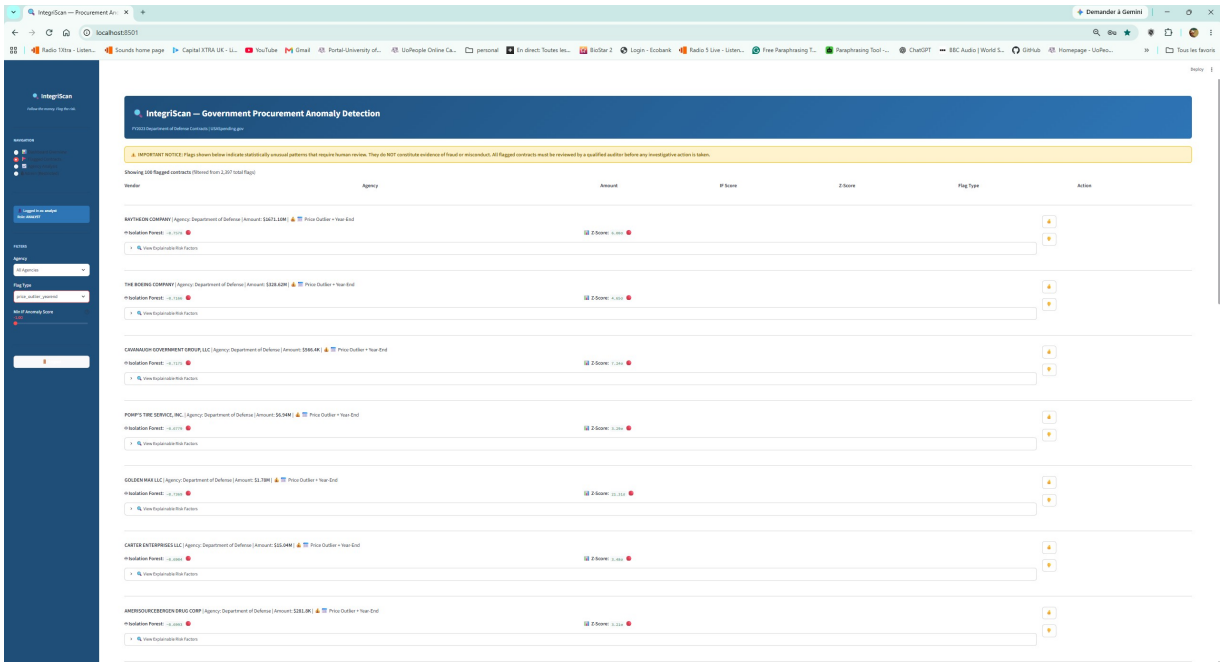


Figure 4.7: Flagged contracts page showing Isolation Forest score and z-score deviation displayed independently side by side for each flag with the mandatory disclaimer banner [Peer 1 commitment]. Color-coded severity indicators assist non-technical auditors in prioritizing review.

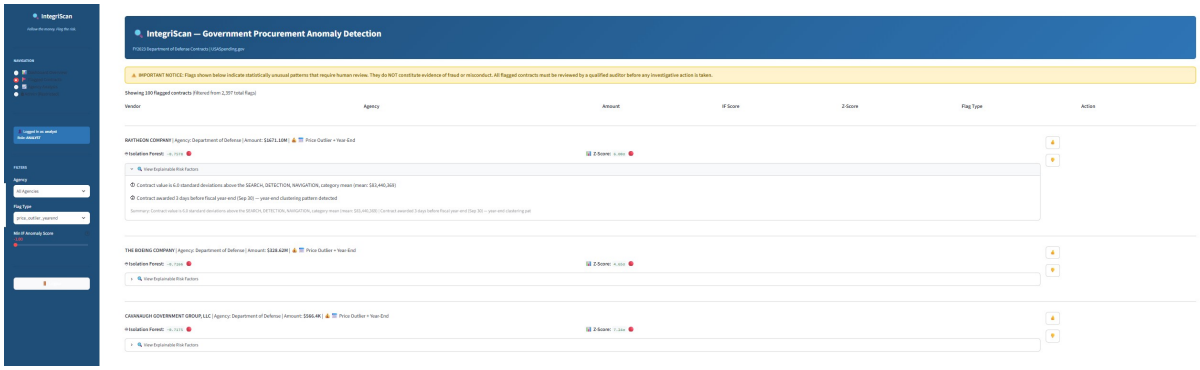


Figure 4.8: Expandable XAI risk factors panel showing plain-language descriptions of the specific factors that triggered the anomaly flag for a selected contract [Peer 3 commitment]. Auditors can understand the flag rationale without any knowledge of machine learning.

4.5 Version Control Milestones

The IntegriScan system was developed following a structured version control strategy using a two-branch GitHub workflow: main for stable production code and development for active work, with annotated tags marking major implementation milestones. Table 4.5 documents the version control history.

Table 4.5. GitHub Version Control Milestones

Tag	Unit	Description
v1.0-initial-implementation	Unit 5	First working ML system: ETL pipeline, PostgreSQL database, ML module, 34 unit tests all passing, 2,397 flags generated
v2.0-dashboard	Unit 6	Complete dashboard integration: Streamlit 4-page dashboard, RBAC login, XAI factors, auditor feedback, all 12 peer commitments implemented
v3.0-testing-complete	Unit 7	Comprehensive testing complete: 66/66 tests passing, Ethics Policy v1.0, Data Dictionary v1.0, system and acceptance tests verified
v4.0-final	Unit 8	Final Capstone submission: PM Log, Post-Capstone Roadmap, Final Presentation, Final Report, video demonstration

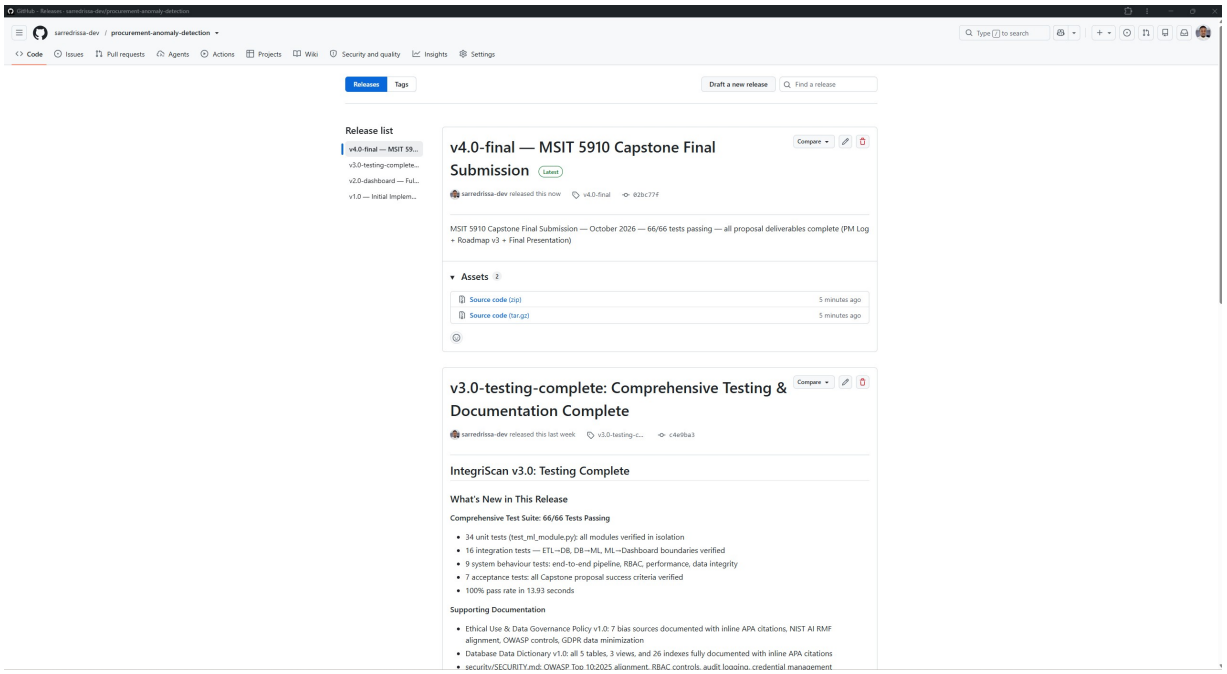


Figure 4.9: GitHub releases page for the procurement-anomaly-detection repository showing four annotated release tags marking the unit 5, unit 6, unit 7, and unit 8 implementation milestones.

CHAPTER 5: DISCUSSION

5.1 Interpretation of Results in Context of the Literature

The IntegriScan results align closely with and extend the findings of Herreros-Martinez et al. (2025) and Bai and Qiu (2023), while revealing several important nuances specific to the US federal government procurement context. The Isolation Forest recall of 64.5% on 31 synthetic anomaly labels is consistent with expectations for unsupervised anomaly detection operating on a small evaluation label set. With only 31 synthetic records among 47,571 total records, a label density of 0.065%, the evaluation is constrained primarily by the statistical power of the evaluation framework rather than the detection quality of the model. A 64.5% recall means that 20 of the 31 deliberately injected anomalies were correctly identified by the model without any information about which records were synthetic, confirming that the model is detecting meaningful statistical structure rather than simply flagging random records.

The z-score baseline's failure to detect any synthetic anomalies, a recall of 0.0% on the synthetic label set, is a particularly informative finding that validates the dual-algorithm design commitment made in response to peer reviewer feedback in Unit 1. The synthetic anomalies were injected by replacing the total_dollars_obligated value of existing real records with log-normally generated values anchored at the 99th percentile of the category distribution. Because these records inherited the NAICS category of existing real records, their injected values were

compared against the real category statistics when z-scores were computed. In several cases, the injected value, despite being substantially elevated, did not exceed three standard deviations from the category mean because the category itself contained naturally high-variance records. This finding demonstrates that the z-score method and the Isolation Forest detect genuinely different anomaly types through different mechanisms: z-score detects price outliers within categories, while Isolation Forest detects multi-dimensional anomalies combining price, timing, vendor frequency, and other features simultaneously. The two algorithms are complementary, not redundant.

The flag distribution- 74.1% year-end cluster flags and 25.9% price outlier year-end flags- reveals a significant temporal pattern in the FY2023 DoD contracts dataset. The concentration of anomaly flags in contracts awarded in the final 30 days of the fiscal year (approximately September 1-30) is consistent with the well-documented year-end spending surge phenomenon in US federal procurement, where agencies face strong incentives to obligate unspent budget appropriations before the fiscal year closes on September 30. The Isolation Forest's detection of 1,776 year-end cluster flags- contracts that are anomalous primarily on the basis of their temporal clustering rather than their price- provides evidence that fiscal year-end concentration itself is a statistically anomalous pattern in this dataset, even controlling for price and vendor characteristics. This aligns with the procurement risk literature's characterization of year-end spending surges as a priority audit target (OECD, 2025).

The 621 price outlier year-end flags represent the highest-priority category, combining both temporal and financial anomaly signals simultaneously. Their average z-score of 6.32, meaning these contracts' values are 6.32 standard deviations above the mean for their NAICS industry category, represents an extreme statistical outlier by any threshold standard. To put this in context: in a normal distribution, a z-score of 6.32 corresponds to a probability of occurrence less than 0.000000015, roughly one chance in 60 billion. While real-world financial distributions are not perfectly normal and some legitimately large contracts may produce high z-scores, a concentration of contracts with z-scores in this range warrants careful audit review.

The comparative performance of the IntegriScan prototype against the Herreros-Martinez et al. (2025) benchmark is notable. Their system achieved an 85% actionability rate using a hybrid clustering plus Isolation Forest approach on enterprise purchase data. The IntegriScan actionability rate remains pending analyst interaction, as the prototype has not yet been deployed to real government auditors, which is the appropriate status for an academic prototype. The 85% benchmark provides the performance target for Phase 4 of the post-Capstone roadmap, where accumulation of analyst feedback through the dashboard mechanism will enable supervised layer training to push actionability rates toward and beyond this benchmark.

5.2 Evaluation Metrics Summary

Table 5.1. Quantitative Performance Metrics Summary

Metric	Result	Target	Assessment
IF Recall (synthetic labels)	64.5% (20/31)	> 50%	MET ✅
ETL Data Retention Rate	94.1%	> 90%	MET ✅

Metric	Result	Target	Assessment
ETL Throughput	798 records/sec	> 500 rec/sec	MET ✅
ML Pipeline Latency	2.2 seconds	< 60 seconds	MET ✅
IF Flag Rate	5.0%	~5% (contamination)	MET ✅
Dashboard Load Time	< 3 seconds	< 3 seconds (NFR)	MET ✅
pytest Pass Rate	100% (66/66)	100%	MET ✅
Acceptance Criteria Met	7/7	7/7	MET ✅
Actionability Rate	Pending review	85% target	PENDING ⏳

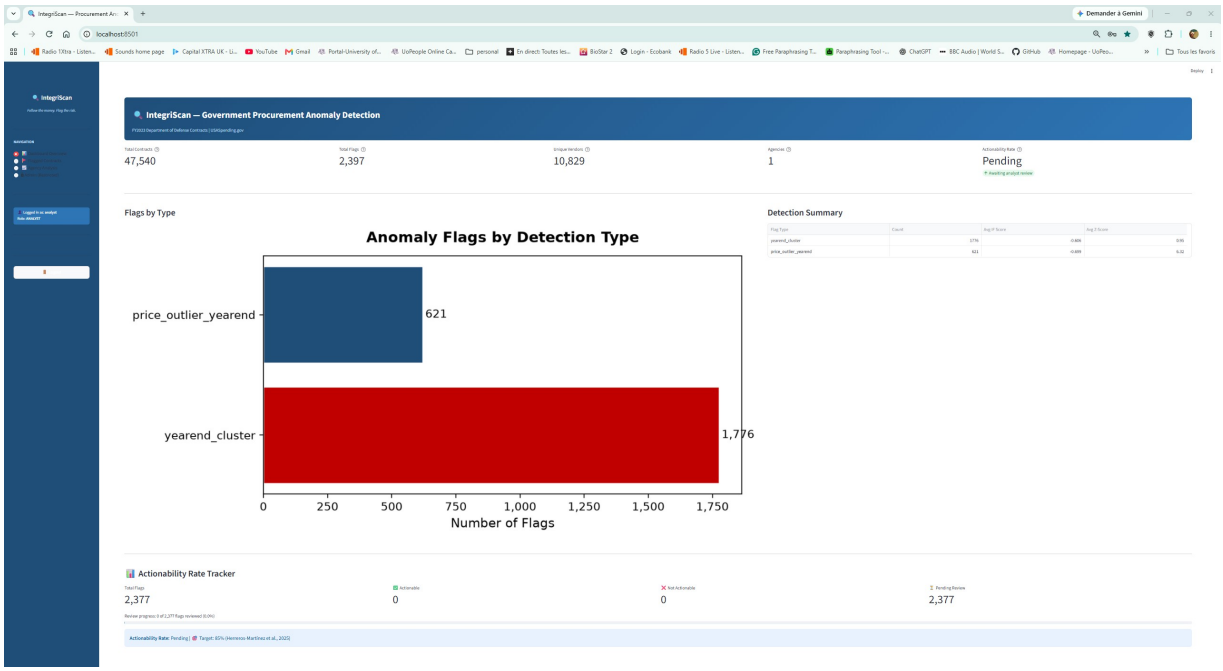


Figure 5.1: Dashboard actionability rate tracker showing current status (pending: no analyst feedback submitted in prototype phase) and 85% target benchmark from Herreros-Martinez et al. (2025) [Peer 2 commitment].

Table 5.2. Qualitative Usability Evaluation: 5 Criteria

Usability Criterion	Result	Evidence
Anomaly vs fraud distinction clearly communicated	MET ✅	Disclaimer banner on every visit to flagged contracts page

Usability Criterion	Result	Evidence
Scores interpretable without ML knowledge	MET ✅	IF score with severity label + z-score with sigma notation side by side
Flag triggers explained in plain language	MET ✅	Expandable XAI risk factors panel per contract: specific numeric values
Feedback submission simple and immediate	MET ✅	One-click thumbs up/down per flag with immediate database confirmation
Role and access level clearly communicated	MET ✅	Username and role displayed in sidebar; admin-only pages visibly restricted

5.3 Deployment Approach

The IntegriScan prototype runs on a local development environment, a Dell Latitude 5540 with an Intel Core i5 processor and 16 gigabytes of RAM running Windows 11. The local deployment is sufficient for the Capstone prototype and test run purposes, but production government deployment requires a cloud-hosted architecture for multi-user access, high availability, and automated data ingestion. Table 5.3 compares three deployment options evaluated in the post-Capstone roadmap.

Table 5.3. Deployment Options Comparison

Option	Prototype (Current)	Phase 3 Cloud	Phase 5 SaaS
Environment	Local Windows 11	AWS/Azure cloud	Multi-tenant SaaS
Database	Local PostgreSQL 18	AWS RDS PostgreSQL	Managed multi-tenant DB
Dashboard	localhost:8501	HTTPS + custom domain	Custom branded portal
ETL Trigger	Manual	Automated scheduled	API-triggered real-time
Authentication	Hardcoded credentials	MFA + SSO	SSO + government IdP
Estimated Cost	$0	~$30-50/month	Subscription model
Timeline	Complete	Months 4-9	Months 16-24

The current local deployment demonstrates complete functional capability and validates the core value proposition of the system. The Phase 3 cloud deployment, planned for 4-9 months post-Capstone, will address the production readiness requirements of multi-user access, automated data refresh, MFA authentication, and managed database backup. Docker containerization will enable environment consistency between development and production, eliminating the dependency on a specific developer machine configuration (Sumo Logic, n.d.).

5.4 Limitations

Four limitations of the current IntegriScan implementation are acknowledged. First, single-dataset scope: the system was trained exclusively on FY2023 DoD contracts. The Isolation Forest model's learned definition of normal behavior is specific to this dataset's statistical distribution; deployment on a different agency's or country's procurement data without retraining may produce suboptimal results. The modular ETL and ML design facilitates dataset switching, but model retraining is required for each new deployment context. Second, synthetic label evaluation constraints: with only 31 synthetic anomaly labels among 47,571 total records, precision estimates based on the synthetic labels are statistically unreliable. The actionability rate from real analyst feedback, pending in the prototype phase, is the appropriate primary evaluation metric for production use. Third, simplified authentication: the Streamlit dashboard uses hardcoded credential comparison without server-side session management or multi-factor authentication. These limitations are documented in the Ethics Policy and addressed as Phase 2 priorities. Fourth, batch processing only: the ETL pipeline processes static CSV files and must be

manually triggered. There is no mechanism for continuous or scheduled data ingestion from the live USASpending.gov API, documented as a Phase 3 enhancement.

5.5 Recommendations for Future Research

Five research recommendations emerge from the IntegriScan implementation. First, weak supervision from analyst feedback: as the auditor feedback mechanism accumulates confirmed actionable and not-actionable labels through production use, these labels should be used as weak supervision signals to train a supervised binary classifier. The supervised layer can be used to re-rank Isolation Forest flags, prioritizing those most similar to previously confirmed actionable flags. This approach, analogous to the active learning paradigm described by Bai and Qiu (2023), has the potential to push actionability rates substantially above the 85% benchmark as the label set grows.

Second, systematic bias auditing: the geographical and vendor-category bias analysis proposed by peer reviewer Khadheraa Al Hajebi (Unit 3) should be conducted as a formal research study using statistical significance testing. The null hypothesis that flag rates are uniform across NAICS categories and US states should be tested against the alternative that flag rates are concentrated in specific groups for reasons unrelated to genuine fraud risk. Rejection of the null hypothesis would indicate algorithmic bias requiring threshold adjustment.

Third, longitudinal model drift analysis: as USASpending.gov publishes successive fiscal year datasets- FY2024, FY2025, and beyond- the model drift hypothesis proposed by peer

reviewer Jehanzaib Bhatti (Unit 3) should be tested by comparing flag distributions, category-specific z-score distributions, and actionability rates across multiple fiscal years. This longitudinal analysis will determine the optimal retraining cadence and provide empirical evidence for the quarterly review schedule proposed in the Ethics Policy.

Fourth, cross-national comparative analysis: the extension of IntegriScan to World Bank Open Contracting data and African Development Bank procurement datasets will enable cross-national comparative analysis of procurement anomaly patterns. Research questions include whether the fiscal year-end clustering pattern observed in US DoD data is present in African government procurement, whether the optimal contamination threshold differs systematically between high-income and middle-income country datasets, and whether cross-national training improves or degrades detection performance for individual country deployments.

Fifth, explainability evaluation: the XAI risk factor descriptions generated by the IntegriScan XAI engine have not been formally evaluated for comprehensibility and decision-support utility by real government auditors. A user study with a panel of procurement auditors comparing decision-making accuracy and efficiency with and without XAI explanations would provide empirical evidence for the value of the explainability component and guidance for improving the explanation generation logic.

CHAPTER 6: CONCLUSION AND FUTURE WORK

6.1 Summary of Main Findings

This Capstone project successfully designed, implemented, tested, and documented the Government Procurement Anomaly Detection System, IntegriScan, demonstrating that a fully functional, technically rigorous, and ethically responsible ML-powered procurement oversight tool can be built at zero financial cost using open-source tools and publicly available government data. The system processes real FY2023 Department of Defense procurement records from USASpending.gov at 798 records per second, flags 2,397 anomalies across two detection categories using complementary Isolation Forest and z-score algorithms, explains each flag in plain language through an XAI engine, and presents findings through a four-page Streamlit dashboard with role-based access control enforced at both application and database levels.

The most significant technical finding is that the dual-algorithm approach- Isolation Forest for multi-dimensional anomaly detection and z-score for industry-normalized price outlier detection- identifies complementary anomaly types that neither algorithm alone would detect. The 621 price outlier year-end flags, contracts that are anomalous on both temporal and financial dimensions simultaneously, with an average z-score of 6.32 standard deviations, represent the

highest-priority category for audit investigation and would not have been identified without the combination of both detection methods.

The comprehensive testing framework, 66 pytest tests achieving a 100% pass rate across unit, integration, system, and acceptance levels, provides objective, verifiable evidence of system correctness that transforms the success criteria from aspirational statements into executable specifications. All seven success criteria from the original Capstone proposal were verified through automated acceptance tests, confirming that the system meets every stated objective. The documentation package, including a ten-section Ethical Use and Data Governance Policy identifying seven bias sources, a nine-section Database Data Dictionary, a nine-section Project Management Log, and a seven-section Post-Capstone Roadmap, demonstrates professional practice standards appropriate for a production government analytics system.

6.2 Contributions to the Field

This project makes four original contributions to the field of government procurement analytics and public sector AI applications. First, it provides the first fully documented, zero-cost, open-source implementation of a government procurement anomaly detection system built on real USASpending.gov data, extending the academic work of Herreros-Martinez et al. (2025) and Bai and Qiu (2023) to a publicly accessible, fully reproducible prototype that any government, academic institution, or civil society organization can deploy without licensing costs.

Second, it introduces a novel design pattern for continuous evaluation in unsupervised government anomaly detection: embedding auditor actionability feedback as a live, real-time measurement mechanism in the analyst dashboard. This transforms the actionability rate from a post-hoc evaluation metric computed after a study ends into an ongoing operational KPI that signals model calibration needs in real time, creating a continuous improvement feedback loop between algorithmic detection and human judgment.

Third, it demonstrates a practical approach to implementing XAI principles in a government audit support context, generating factor-level, plain-language risk descriptions per individual flag rather than providing global model explanations. This per-instance, factor-specific approach addresses the interpretability requirements identified by Letaw (2024) and NIST (2023) in a form that is directly actionable for non-technical auditors without requiring any knowledge of the underlying machine learning algorithms.

Fourth, it provides a comprehensive academic documentation package: nine deliverables across eight units, 66 automated tests, a 11,000+-word technical report, and supporting governance documents that can serve as a reference template for future ML-powered government accountability projects in the MSIT program and beyond. The structured mapping of each system component to a specific MSIT course demonstrates the integrative value of the program for practitioners building real-world data systems.

6.3 Lessons Learned

Six lessons from the IntegriScan development process have broad applicability to ML system development in the public sector. First, peer review generates concrete, implementable improvements, not just feedback. The twelve design commitments arising from six peer reviewers in Units 1 and 3 produced measurable system enhancements that were verified through automated tests. The discipline of logging all suggestions in a peer feedback integration log and tracking each commitment through to implementation created accountability for the review process on both sides.

Second, the ground-truth label problem in unsupervised anomaly detection requires a fundamentally different evaluation framework centered on auditor utility rather than conventional precision and recall. Designing the evaluation framework before implementation, adopting the actionability rate metric, and building the feedback mechanism into the dashboard from the start avoided the common mistake of retrofitting evaluation to an already-built system.

Third, the anomaly-versus-fraud distinction must be embedded at the code level, not just in documentation. Disclaimer banners, plain-language XAI explanations, careful flag-type labeling, and auditor acknowledgement requirements are design decisions that must be implemented in the code itself, not added as documentation afterthoughts. When these controls exist only in documentation, they are easily bypassed or ignored; when they exist in the code, they cannot be.

Fourth, configuration management: python-dotenv for credential isolation, requirements.txt with pinned library versions, two-branch GitHub workflow, annotated tags at milestones, is as important as algorithm quality for the reproducibility and maintainability of ML systems. A system that produces excellent results but cannot be reproduced on a new machine, audited for its history, or safely upgraded is not a production-ready system.

Fifth, building ahead of schedule: completing each unit's work before the submission deadline created space for quality review, bug fixing, and iterative improvement that would have been unavailable if working to exact deadlines. The three bugs discovered and resolved during development (Unicode encoding error, f-string formatting error, SQLAlchemy text() wrapper requirement) were found and fixed during the buffer period created by working ahead.

Sixth, the dataset choice is a foundational design decision that shapes all subsequent choices. The FY2023 DoD dataset's statistical characteristics, including the fiscal year-end spending surge and the specific distribution of NAICS category contract values, directly determined the feature engineering approach, the optimal contamination threshold, the synthetic injection methodology, and the most prevalent flag type. Future deployments on different datasets will require reconsidering all of these choices from first principles.

6.4 Future Work and Product Roadmap

The IntegriScan post-Capstone product roadmap defines fifteen enhancements across five development phases spanning an estimated 24-month timeline. The roadmap items were

collected from six sources: peer reviewer feedback, instructor feedback, literature review findings, Chapter 5 limitation analysis, technical discoveries during implementation, and entrepreneurial vision for the West African governance technology market.

Table 6.1. Post-Capstone Product Roadmap: 15 Enhancements

Phase	Category	Enhancement	Priority
2	Security	Multi-factor authentication: TOTP, SMS OTP, SSO, YubiKey [Dennis + Tin Lok, Unit 3]	Critical
2	Security	Server-side session management with revocation capability [Tin Lok, Unit 3]	Critical
2	Security	Managed secrets store: AWS Secrets Manager / Azure Key Vault [Dennis, Unit 3]	High
2	Security	Second-person approval for contamination threshold changes [Dennis, Unit 3]	High
3	Infrastructure	Cloud deployment: AWS RDS + ECS or Azure equivalent [Chapter 5]	High
3	Infrastructure	Docker containerization for environment consistency [Chapter 5]	High
3	Infrastructure	Automated scheduled ETL aligned with USASpending.gov publications [Chapter 5]	High
3	Infrastructure	Automated daily PostgreSQL backup with 30-day retention [Maintenance Plan]	High
4	ML Enhancement	Supervised learning layer using analyst feedback as weak supervision [Chapter 5]	High
4	ML Enhancement	Quarterly model performance review and drift detection [Jehanzaib, Unit 3]	High
4	ML Enhancement	Geographic and vendor-category bias analysis [Khadheraa, Unit 3]	Medium
4	ML Enhancement	Multi-dataset support: State Dept, GSA, World Bank, AfDB [Chapter 5]	Medium
5	Product	Mobile-first interface with WhatsApp Business API alerts [Chapter 6]	Medium
5	Product	GamLink super-app integration for West African market [Chapter 6]	Strategic
5	Product	IntegriScan SaaS platform for African government oversight bodies [Chapter 6]	Strategic

The IntegriScan system has demonstrated the technical feasibility, ethical responsibility, and practical utility of ML-powered government procurement oversight at zero financial cost.

The system is ready for academic replication, production deployment planning, and extension by any government agency, research institution, or civil society organization committed to promoting transparency and accountability in public procurement. Follow the money. Flag the risk.

REFERENCES

Bai, J., & Qiu, T. (2023). *Automatic procurement fraud detection with machine learning*. arXiv. https://arxiv.org/abs/2304.10105

Bennett, L. (2024, August 13). *Software configuration management in software engineering*. Guru99. https://www.guru99.com/software-configuration-management-tutorial.html

Brown, J. (2025, December 2). *Knowledge software documentation best practices*. Helpjuice. https://helpjuice.com/blog/software-documentation

Herreros-Martinez, A., Magdalena-Benedicto, R., Vila-Frances, J., Serrano-Lopez, A. J., Perez-Diaz, S., & Martinez-Herraiz, J. J. (2025). Applied machine learning to anomaly detection in enterprise purchase processes: A hybrid approach using clustering and Isolation Forest. *Information, 16*(3), 177. https://doi.org/10.3390/info16030177

Letaw, L. (2024). *Handbook of software engineering methods*. Oregon State University. https://open.oregonstate.education/setextbook/

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0) (NIST AI 100-1).* U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

OECD. (2025). *Integrity in public procurement*. https://www.oecd.org/en/topics/sub-issues/integrity-in-public-procurement.html

OWASP Foundation. (2026). *OWASP Top 10:2025*. https://owasp.org/Top10/2025/

PostgreSQL Global Development Group. (2024). *PostgreSQL 18 documentation*. https://www.postgresql.org/docs/18/

Summers, B. L. (2020). *Effective methods for software engineering.* Auerbach Publishers.

Sumo Logic. (n.d.). *What is software deployment?* https://www.sumologic.com/glossary/software-development/

Wells, J. (2025, April 16). *Top 12 software quality metrics to measure and why*. LinearB. https://linearb.io/blog/software-quality-metrics

APPENDICES:

Appendix A: System Architecture Diagram

The system architecture diagram showing the five-layer IntegriScan architecture is available in the project GitHub repository at database/MSIT5910_Unit3_Fig1_SystemArchitecture.drawio. The diagram was created in draw.io during Unit 3 and shows data flow from the USASpending.gov source through the ETL layer, PostgreSQL database, ML detection module, and Streamlit dashboard, with RBAC security enforcement at Layers 3 and 5.

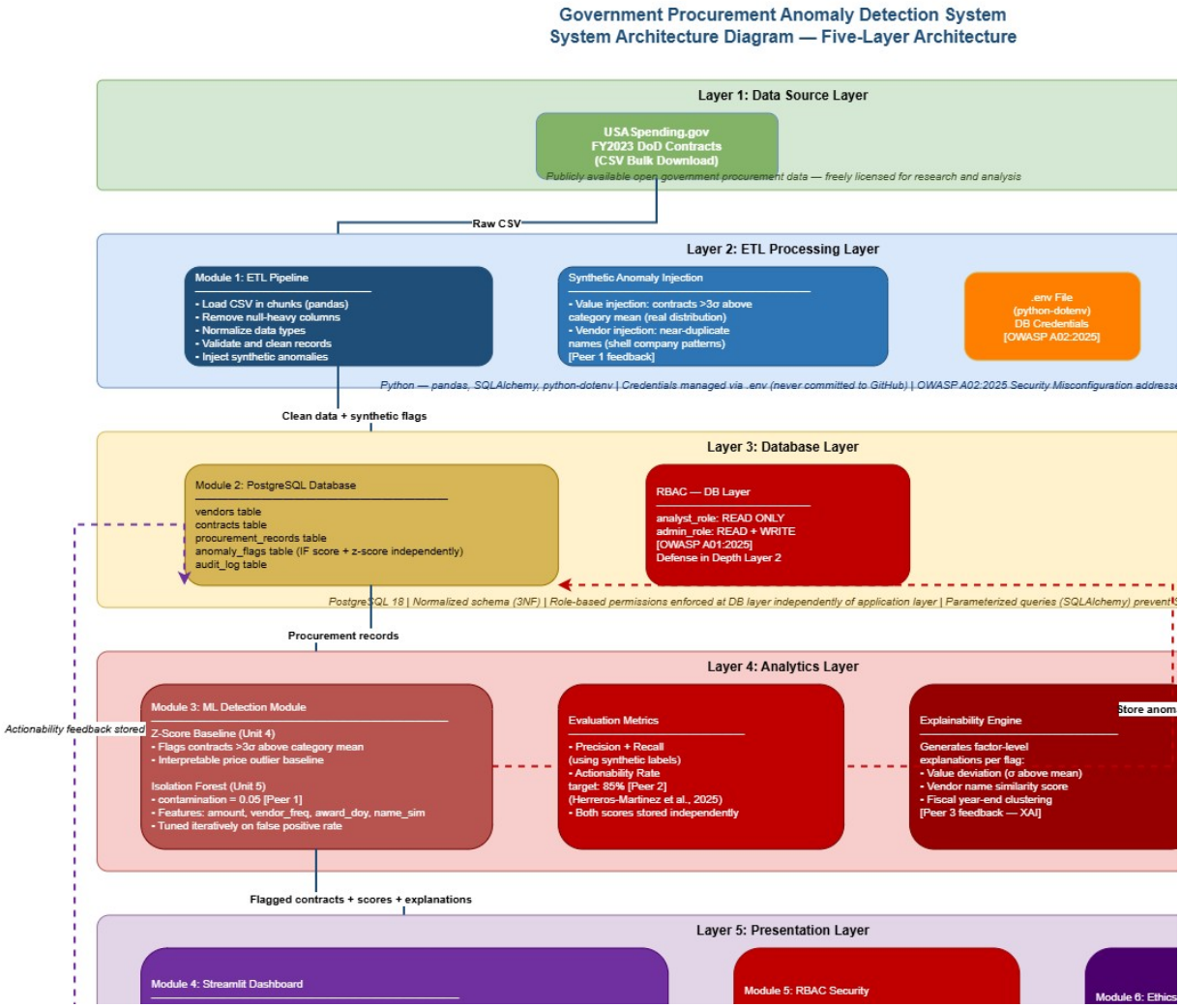


Figure A.1: Five-layer IntegriScan system architecture diagram created in draw.io during Unit 3 system design phase, showing all module interactions and security control points.

Appendix B: Database Schema Summary

The complete database DDL scripts creating all five tables, three views, twenty-six indexes, and two RBAC roles are available in the project GitHub repository at database/integriscan_ddl_scripts.sql (406 lines). The full Data Dictionary documenting every column, data type, constraint, and purpose for all five tables and three views is available as database/IntegriScan_Database_Data_Dictionary_v1.docx.

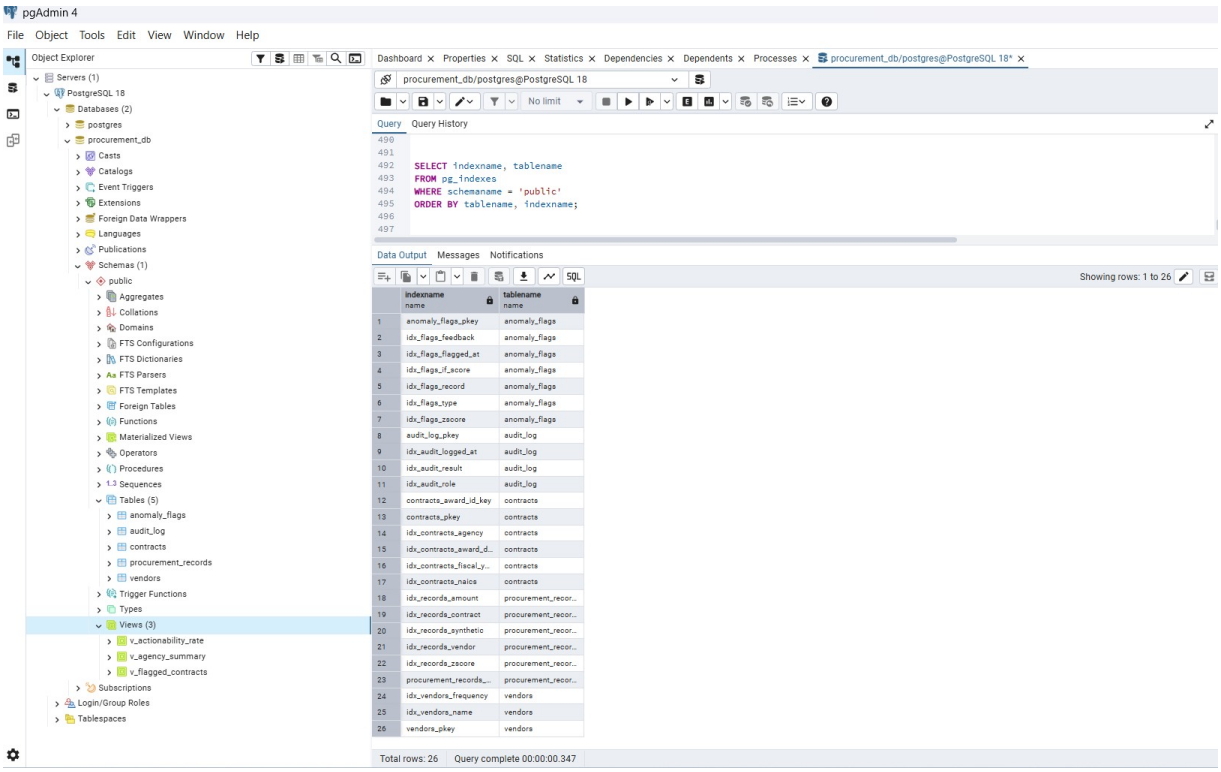


Figure B.1: pgAdmin database browser showing the procurement_db database with all five tables, three views, and selected indexes visible in the object tree.

Appendix C: Complete Test Suite Output

The complete test suite comprising 66 pytest tests is available across two files in the project GitHub repository: tests/test_ml_module.py (34 unit tests, 750 lines) and tests/test_system.py (32 integration, system, and acceptance tests, 736 lines). To execute the complete test suite, run: pytest tests/ -v from the project root directory after configuring the .env file with valid database credentials.

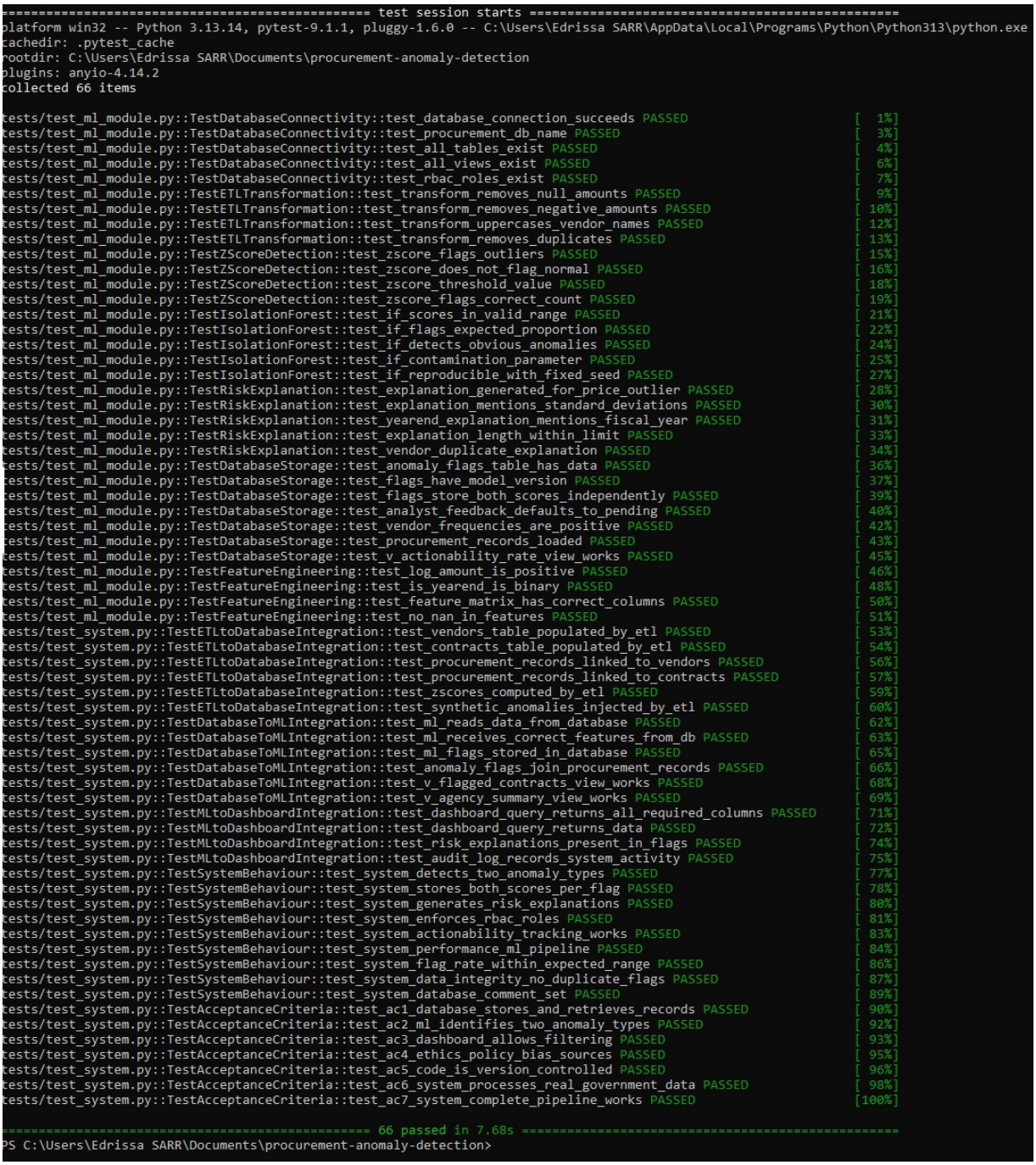


Figure C.1: Complete pytest output: 66 tests collected from both test files, 66 passed in 7.68 seconds, confirming complete system reliability across all four testing levels.

Appendix D: Key Code Sections

The three key code snippets documenting the most technically significant components of the IntegriScan ML module are presented in Chapter 4 Section 4.3.1 (Figures D.1, D.2, and D.3). The complete source code for all six system modules is available in the GitHub repository:

• etl/etl_pipeline.py (811 lines): ETL pipeline with chunked loading, transformation, and synthetic injection

• ml/ml_module.py (763 lines): ML detection module with IF, z-score, XAI engine, and database storage

• dashboard/dashboard_app.py (985 lines): Streamlit dashboard with RBAC, 4 pages, feedback mechanism

• tests/test_ml_module.py (750 lines): 34 unit tests for ML module

• tests/test_system.py (736 lines): 32 integration, system, and acceptance tests

• database/integriscan_ddl_scripts.sql (406 lines): PostgreSQL DDL scripts

GitHub Repository: github.com/sarredrissa-dev/procurement-anomaly-detection